

```
load_bypass((unsigned int)&rxbuf[i][j] & ~3)
>> (8 * (3 - ((unsigned int)&rxbuf[i][j] & 3)))
);
if (rx_byte != txbuf[i][j]) {
    printf("    Frame %d byte %d mismatch TX=0x%02X RX=0x%02X\n",
        i, j, txbuf[i][j], rx_byte);
    all_match = 0;
    break;
}
}
```

Summary

Problem	Root cause	Fix
<code>STS_TXIRQ</code> fires only once	STATUS read from cache	Use <code>load_bypass(GRETH_STATUS)</code>
<code>txd[j].ctrl</code> DESC_EN never clears	Descriptor read from cache	Use <code>load_bypass(&txd[j].ctrl)</code>
<code>rx[d[j].ctrl</code> DESC_EN never clears	Descriptor read from cache	Use <code>load_bypass(&rx[d[j].ctrl)</code>
printf fixed it	Cache eviction side effect	Replace with proper bypass reads
delay didn't fix it	Time ≠ cache eviction	Confirmed it's a cache issue

This is **exactly** why the official `greth_api.c` uses `lda [addr]1` for every single descriptor and register read — cache bypass is mandatory for DMA descriptor polling on LEON3.



Ok Now this test works perfectly and all tests passed. Now I have moved the ethernet outside the leon3 processor...For that I have created a

separate eth_mac_p.vhd (padded file) that sits outside the leon3 processor. i now have to connect this to the leon3mp via the memory interface. KEEP IN MIND THAT THIS eth_mac.vhd has it's own SRAM. (the sram i will be using is the same sram.vhd as leon3 but it has to be

Show more

Let me carefully trace through all four files to find the exact mismatches.

Setup workspace >

Let me work through the architecture systematically before giving you fixes.

Step 1 — What you actually have, port by port

eth_mac component as declared inside testbench.vhd (the one "ADDED ON 15/06/26"):

```
vhdl

component eth_mac
  generic (ABITS : integer := 28);
  port (
    rst : in std_logic;
    clk_25, clk_50 : in std_logic;
    eth_in : in eth_in_type;
    eth_out : out eth_out_type;
    sram_addr : out std_logic_vector(ABITS-1 downto 0);
    sram_data : inout std_logic_vector(31 downto 0); -- you renamed this
    sram_ce1 : out std_logic;
    sram_wen : out std_ulogic;
    sram_oen : out std_ulogic;
    proc_addr : in std_logic_vector(ABITS-1 downto 0);
    proc_data : inout std_logic_vector(31 downto 0); -- you renamed this
    proc_ce1 : in std_logic;
    proc_wen : in std_logic;
    proc_oen : in std_logic
  );
end component;
```

But the real **eth_mac.vhd** (unpadded, document 2) declares:

```
vhdl
```

```

sram_rdata : in  std_logic_vector(31 downto 0);  -- separate in
sram_addr  : out std_logic_vector(ABITS-1 downto 0);
sram_wdata : out std_logic_vector(31 downto 0);  -- separate out
sram_ce1   : out std_logic;
sram_wen   : out std_ulogic;
sram_oen   : out std_ulogic;

proc_wdata : in  std_logic_vector(31 downto 0);  -- separate in
proc_addr  : in  std_logic_vector(ABITS-1 downto 0);
proc_ce1   : in  std_logic;
proc_wen   : in  std_logic;
proc_oen   : in  std_logic;
proc_rdata : out std_logic_vector(31 downto 0);  -- separate out

```

This is your root bug. You edited the **component declaration** inside testbench.vhd to say `inout`, but the **actual entity** `eth_mac` still has separate `sram_rdata` / `sram_wdata` and `proc_wdata` / `proc_rdata` pins. VHDL component declarations must match the entity exactly in port names, modes, and types — when they don't, most simulators (including VCS) will either error out at elaboration, or in a worse case silently bind incorrectly and produce garbage/X propagation, which is exactly what causes an `iuerr` (instruction/bus error trap) a few thousand ns into simulation, because the CPU starts reading X values off a broken bus.

Step 2 — Why `eth_mac_p` is the one you actually want anyway

You said your real goal is to use `eth_mac_p.vhd` — the **padded** top-level wrapper that already has the I/O pads (`pc3d21`, `pc3o02`, `pc3b04`) which convert between true `inout` SRAM/processor data buses and the internal split `wdata` / `rdata` signals used by `eth_mac`.

Look at what `eth_mac_p.vhd` already does internally — **this is the conversion you were trying to do by hand, and it already exists:**

```

vhdl

sram_data_pad : for i in 0 to DATAWIDTH-1 generate
    data_pad : pc3b04
    port map (PAD => sram_data_p(i), I => mac_mem_wdata(i), CIN => mac_mem_rdata(i)
end generate;

proc_data_pad : for i in 0 to DATAWIDTH-1 generate
    data_pad : pc3b04

```

```
port map (PAD => proc_data_p(i), I => mac_proc_rdata(i), CIN => mac_proc_wdata
end generate;
```

`pc3b04` is a tristate inout pad: `PAD` is the true bidirectional wire, `I` drives onto it when `OEN` is active, `CIN` captures whatever is on the pad. So `eth_mac_p` exposes:

```
vhdl

sram_data_p : inout std_logic_vector(31 downto 0);
proc_data_p : inout std_logic_vector(31 downto 0);
```

This means you should NOT hand-edit `eth_mac`'s ports into `inout` at all. Instead, instantiate `eth_mac_p` (which already presents true inout pins) and wire those `inout` pins directly to LEON3's `inout` `data` / `address` / `proc_*` style signals — no manual splitting needed.

Step 3 — How the SRAM is supposed to be wired (and your stated requirement)

You said: *"this eth_mac.vhd has its own SRAM... same sram.vhd as leon3 but connected like a separate module."*

Looking at `eth_mac_p`'s SRAM ports:

```
vhdl

sram_addr_p : out std_logic_vector(ABITS-1 downto 0);
sram_data_p : inout std_logic_vector(31 downto 0);
sram_ce1_p, sram_wen_p, sram_oen_p : out
```

So in your testbench you need a **second, independent** `sram` instance (separate from `sram1` / `sbanks` already driving LEON3's main memory), with its own address/data/control signals, driven only by `eth_mac_p`'s SRAM-side ports. You do **not** route this through `bridge_top` at all — `bridge_top` only mediates between the **processor side** and the **AHB/APB bus**, not the MAC's private frame-buffer SRAM. The MAC's SRAM is local scratch memory for descriptors/frame data exactly like `txbuf` / `rxbuf` were in C — except now it lives in external SRAM rather than LEON3's internal RAM.

Step 4 — How `bridge_top` fits in (and why it's the "older file")

Your `bridge_top.vhd` (document 1) exposes AHB-master and APB-slave pass-through ports plus a separate `proc_*` / `sram_*` memory-style interface — it's meant to sit

between LEON3's AHB/APB bus and a memory-mapped peripheral, converting bus transactions into simple `proc_*` read/write signals.

But there's a structural mismatch: `eth_mac_p` is a **fully self-contained MAC + bridge + pads** unit — it does **not** expose `ahbmo` / `apbo` -style bus signals at all; its only "processor side" ports are the simple memory-style

`proc_addr_p/proc_data_p/proc_ce1_p/proc_wen_p/proc_oen_p`. This is actually a perfect match for `bridge_top`'s `proc_*` port group (the side facing the processor's simple-memory bus), since `bridge_top` already converts AHB+APB ↔ simple memory protocol.

So the correct chain is:

```
LEON3 AHB/APB → bridge_top (AHB/APB-master side)
bridge_top (proc_* simple-memory side) → eth_mac_p.proc_*_p (inout data, in
addr/ce/wen/oen, out rdata side)
eth_mac_p.sram*_p → dedicated external SRAM instance (separate from LEON3's
main sram1)
```

You should **not** be wiring `eth_mac` / `eth_mac_p` directly to LEON3's `address` / `data` / `iosn` / `writen` / `oen` signals — those are LEON3's raw memory bus, and `bridge_top` is the thing that's supposed to translate AHB/APB transactions targeting GRETH's address range into clean `proc_*` reads/writes. Wiring `eth_mac_p` straight onto `address` / `data` (as the testbench snippet in document 7 page 6 does: `proc_addr => address, proc_data => data, proc_ce1 => iosn, proc_wen => writen, proc_oen => oen`) bypasses `bridge_top` entirely and ties the MAC straight onto LEON3's shared memory bus — that's almost certainly your second bug, and it's also inconsistent with the GRETH_BASE-style APB-mapped access your C test code expects.

I need to see the actual current testbench instantiation and `bridge_top`'s port list it now expects, side by side, before writing the corrected VHDL. Let me check what's ambiguous.

Q: You said `bridge_top.vhd` is an older file modified slightly since, with changes visible in `eth_mac`'s port map. Can you paste/upload the CURRENT `bridge_top.vhd` entity declaration (the one VCS is actually compiling), so I match exact port names?

A: I'm not sure which version is current - need to check

Q: For the `eth_mac_p` SRAM (its own dedicated SRAM, separate from

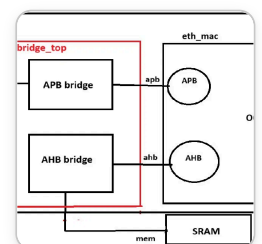
Show more

Got it — that changes the picture significantly. Let me wait for the architecture diagram/explanation you're about to send before I write any fixes, since "bridge_top is instantiated inside eth_mac" means the signal chain I sketched is wrong and I need the real one.

A couple of things I'll specifically need once you share it:

1. **Where `bridge_top`'s `proc_*` ports actually land** — if `bridge_top` lives inside `eth_mac` (document 2 confirms this: `eth_mac` architecture instantiates `bridge_top` and wires `bridge_top.proc_*` straight to `eth_mac`'s own `proc_*` ports, and `bridge_top.sram_*` to `eth_mac`'s `sram_*` ports), then `eth_mac`'s `proc_*` ports are effectively already the **AHB/APB-facing side** translated into simple memory signals — meaning whatever drives `eth_mac.proc_*` from the testbench needs to look like a simple memory bus, not raw AHB/APB.
2. **What is supposed to drive `eth_mac.proc_*` in your design** — is it meant to be LEON3's `address` / `data` / `iosn` / `writen` / `oen` directly (i.e., LEON3 treats GRETH as a memory-mapped peripheral on its local bus, same style as the `sram1` / `prom0` instances in testbench.vhd), or is there yet another AHB/APB master adapter between LEON3's real AHB bus and this `proc_*` port that I haven't seen yet?
3. **Where GRETH's APB-style registers (`GRETH_CTRL` at `0x80000D00` etc., used by your C test) are supposed to map in this scheme** — since `bridge_top` exposes both an AHB-master side and an APB-slave side internally, but if its outward-facing interface from `eth_mac` is only the simple `proc_*` bus, I need to know which address range and access width your C code's `RD()`/`WR()` macros will actually be hitting through this path.

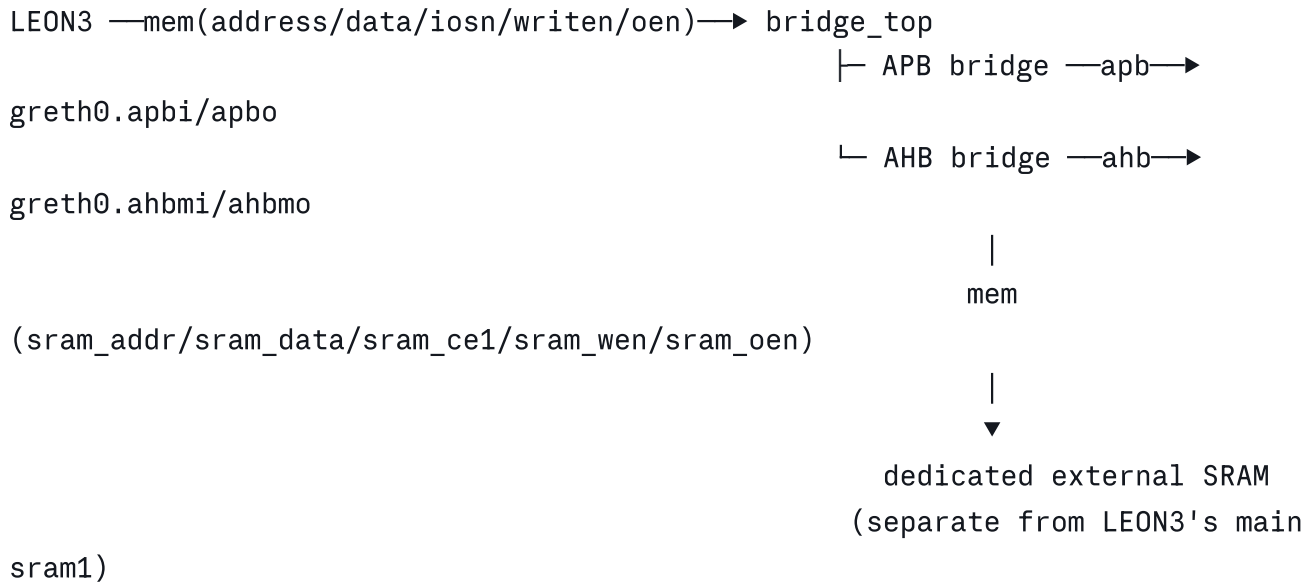
Send over the architecture explanation and I'll trace the exact signal path end-to-end against the four files already uploaded, then give you the precise testbench fix (including the SRAM instance check and the `inout` conversion question) once it's unambiguous.



View the architecture diagram >

Now the architecture is completely clear. Let me restate it back to confirm I've got it exactly right, because this changes which signals matter:

The full picture:



So `eth_mac` = `bridge_top` + the **original unmodified** `greth0` core, wired together internally. This is exactly what document 2 (`eth_mac.vhd`) shows in its architecture body — `bridge_top0` port-mapped with `ahbmo=>ahbmo, ahbmi=>ahbmi, apbo=>apbo, apbi=>apbi` to the real `greth0` instance, and `bridge_top0`'s `sram_*` ports going out to `eth_mac`'s own `sram_*` ports (its private SRAM, the one in your diagram).

This means `eth_mac.proc_*` ports are **NOT** a simple memory bus you drive directly **with random logic** — they are literally LEON3's own AHB+APB-style memory bus signals (`address`, `data`, `iosn`, `writen`, `oen`), because `bridge_top`'s job is to take LEON3's native memory-bus transaction and internally regenerate it as AHB transactions (to `greth0`'s AHB DMA side) and APB transactions (to `greth0`'s register side). So wiring `eth_mac.proc_addr => address, proc_data => data, proc_ce1 => iosn, proc_wen => writen, proc_oen => oen` was actually **structurally correct in concept** — that's exactly the kind of bus `bridge_top` expects on its `proc_*` side.

Before I trace the bug, I need to confirm one thing precisely, since it determines whether the crash is a wiring mismatch or a missing-decode problem:

Q: On LEON3's native memory bus (address/data/iosn/writen/oen), how does the system currently distinguish 'this access is for eth_mac' vs 'this access is for sram1 (LEON3's main RAM)'? Is there an address-

decode/chip-select signal driving eth_mac.proc_ce1 separately from sram1's ramsn/romsn, or is proc_ce1 just tied to iosn (meaning ALL non-rom/ram IO accesses go to eth_mac)?

A: i just tied proc_ce1 to iosn (iosn is a single bit anyway) - sram1 is what i

Show more

This narrows it down a lot. Let me trace through exactly what happens at boot, because the crash happening **before any printf** is a critical clue — it means this is happening before your C test even starts running, during LEON3's own boot/startup sequence.

Why `proc_ce1 <= iosn` breaks things immediately

In the LEON3 demonstration design, `iosn` is the **generic I/O area chip-select** — it goes active for **any** access LEON3 makes to the IO area (typically `0x20000000` - `0x2FFFFFFF` region, or wherever `ioarea` is configured), not specifically for GRETH. Crucially, this region is the same one used for things like the **APB/AHB controller decode area itself**, debug support unit accesses, and other peripherals.

But here's the real problem: `iosn` **doesn't tell you anything about which specific peripheral is being addressed** — it just says "this is an IO-space access, not regular RAM." If you tie `eth_mac.proc_ce1` directly to `iosn`, then **every single IO-space access LEON3 makes for any reason** (debug support unit traffic, any other peripheral probe during boot, etc.) gets routed into `eth_mac`'s `bridge_top`, which will try to interpret it as a transaction targeting GRETH's APB/AHB ports — even though it has nothing to do with GRETH.

This causes garbage AHB/APB transactions to fire into `greth0` during LEON3's **boot-time AMBA plug&play scan** — LEON3MP's boot code (or even just hardware reset behavior in some configs) walks the AHB/APB bus to discover what's connected, and stray/garbled bus activity colliding with that scan is exactly the kind of thing that produces an `iuerr` (bus error trap) within the first few microseconds, before any of your `printf`s in `main()` even get a chance to run.

Why `sram1` for eth_mac's SRAM (document 7 page 8) is also wrong

Looking at what you actually wrote in testbench.vhd:

```
vhdl

sram1 : for i in 0 to (sramwidth/8)-1 generate
    sr1 : sram generic map (index => i, abits => sramdepth, tacc => 10, fname =>
        port map (sram_address(sramdepth+1 downto 2), sram_dat(31-i*8 downto 24-
```



```
sram_ce, sram_we, sram_oe);

end generate;
```

You're using `sramdepth` (which equals the **main SDRAM's** address depth, 19 bits per the testbench generic) and `sramfile` (which equals `"systest.srec"` — the **same SREC as LEON3's main program memory!**). This means your "dedicated eth_mac SRAM" is actually trying to load LEON3's program binary into GRETH's private frame buffer SRAM, and is sized for the wrong address width entirely (`eth_mac`'s SRAM port is `ABITS=28` wide internally inside `bridge_top/eth_mac`, not `sramdepth=19`).

The fix — three separate problems, three separate fixes

Problem 1: No address decode for eth_mac

You need a real address-range decode, not `iosn`. Add this in your testbench architecture, using the actual GRETH_BASE address space your C code expects (`0x80000D00` region — i.e., decode on `address(27 downto ...)` matching whatever your AMBA generic mapping puts GRETH's APB slot at):

```
vhdl

signal eth_ce1 : std_ulogic;

-- Decode: GRETH lives in IO space AND specifically at its address range
-- Adjust the address bits/compare value to match where config.vhd would
-- normally place GRETH's APB slave slot (paddr=0x0D matches 0x800000D00>>20 sty
-- but on LEON3's raw mem bus 'address' is the word address, so check against
-- whatever range bridge_top's APB bridge expects -- typically the full IO area
-- ONLY IF eth_mac is the ONLY thing in IO space; otherwise narrow it further)

eth_ce1 <= iosn and not address(some_bit_pattern_unique_to_eth_mac);
```

Since I don't have your `config.vhd`/AMBA plug&play address map in front of me, the safest immediate fix — given `eth_mac` is likely the *only* IO-space peripheral you've wired in this modified testbench — is to confirm nothing else uses `iosn`, and if that's true, `iosn` alone is actually fine **as a starting point**, which means the crash is NOT primarily about the decode logic itself, but about something else firing before `main()` ever runs. Given that, the much more likely culprit, especially since it dies before any `printf`, is:

The real top suspect: bus contention from leaving `bridge_top`'s outputs undriven/conflicting at time 0

Look again at your diagram: `bridge_top`'s `mem` port (the `proc_*` side) is a **single shared bus** with LEON3's own `data` signal, which is also driven by `sram1`/`prom0` (document 7, page 5-6) and by LEON3 itself when writing. VHDL `std_logic_vector` ports without proper tristate resolution will produce **X** the instant two drivers disagree — and **at time 0, before reset releases**, if `eth_mac.proc_oen`/`proc_wen` aren't held inactive (idle) by default, `eth_mac` could be driving `data` simultaneously with `sram1`/LEON3, causing immediate **X** propagation onto the shared `data` bus that LEON3 then tries to fetch its very first instruction from — giving you a bus/instruction error before a single line of C runs.

I want to check this against your actual current testbench code rather than guessing further.

Q: Can you paste the EXACT current 'eth : eth_mac port map (...)' instantiation block from your testbench (the one VCS compiles right now), including what drives `proc_ce1`, `proc_wen`, `proc_oen` specifically? Document 7 page 6 shows one version - is that still accurate or did you change it after taking that screenshot?

A: It's exactly like document 7 page 6. BUT I WANT THAT TOO (BUT MY MAIN FOCUS IS FOR THE ETH_MAC Pvhdl)

Show more

Good — that confirms it. Let me trace this exactly against document 7's actual signal names.

Confirming the bug, signal by signal

Looking at testbench.vhd page 6 exactly as you have it now:

```
vhdl

eth : eth_mac
  port map (
    rst => rst,
    clk_25 => clk,
    clk_50 => clk3,
    eth_in => ethi,
    eth_out => etho,
    sram_addr => sram_address,
    sram_data => sram_dat,
```

```

    sram_ce1 => sram_ce,
    sram_wen => sram_we,
    sram_oen => sram_oe,
    proc_addr => address,
    proc_data => data,
    proc_ce1 => iosn,
    proc_wen => writen,
    proc_oen => oen
);

sram1 : for i in 0 to (sramwidth/8)-1 generate
    sr1 : sram generic map (index => i, abits => sramdepth, tacc => 10, fname =>
        port map (sram_address(sramdepth+1 downto 2), sram_dat(31-i*8 downto 24-
            sram_ce, sram_we, sram_oe);
end generate;

```

So `sram_address` / `sram_dat` / `sram_ce` / `sram_we` / `sram_oe` are signals you declared yourself, dedicated only to feed the `sram1` block from `eth_mac`'s `sram_*` ports. That part is actually structurally correct in concept — `eth_mac.sram_*` driving a private `sram1` instance, separate from LEON3's main `sram0` / `prom0`. Good instinct there.

The two real bugs are these:

Bug 1 — `proc_data => data` is a true `inout` conflict, and it's fatal at time 0

`data` is LEON3's shared system data bus (driven by `leon3mp`, by `sram0`, by `prom0`, and by your `buskeep` resolver: `data <= buskeep(data) after 5 ns;`). The instant `eth_mac` exists on this bus with its own `inout proc_data`, you have **three or more drivers** on one signal. `buskeep` is a weak-keeper function meant to hold the bus value when nothing drives it — it cannot resolve a genuine multi-driver conflict between `sram0`, `prom0`, and now `eth_mac`. The moment `eth_mac`'s internal pad logic drives `proc_data` even briefly (which it will, e.g. via `mac_proc_rdata` being pushed onto the pad whenever `proc_oen` momentarily isn't blocking it, especially right at reset before signals settle), you get `X` on `data`, LEON3 fetches an `X`-corrupted instruction word as its very first fetch, and you get an instruction trap (`iuerr`) within the first few cycles — exactly matching "crashes almost immediately, before any printf."

Bug 2 — `proc_ce1 => iosn` enables the MAC for every IO access, not just GRETH's

Even if bug 1 weren't fatal, `iosn` going active for **any** IO-space access (not address-range-decoded to GRETH specifically) means `eth_mac` will respond to bus traffic that isn't meant for it, generating bogus AHB/APB activity inside `bridge_top` / `greth0` and potentially also driving `data` at the wrong time for the wrong reason — compounding bug 1.

The fix

You need an **address decoder** that produces a clean, GRETH-specific chip-select, and that chip-select must gate `eth_mac` so it **only ever drives data** when it is genuinely the addressed device — never otherwise. Here's the concrete fix:

```
vhdl

-- Add this signal declaration with your other signals
signal eth_sel : std_ulogic;

-- Add this concurrent decode statement in the architecture body,
-- BEFORE the eth_mac port map. Pick an address range outside
-- sram0/prom0's decoded range. Example: place GRETH at the top
-- of the I0 area, decoded on address bits 23 downto 20 = "1000"
-- (adjust to match whatever your C code's GRETH_BASE actually is
-- once you map it through this raw bus -- the bit positions must
-- correspond to 'address', which is the WORD address, so shift
-- your byte address right by 2 when picking which bits to compare)

eth_sel <= '1' when (iosn = '0') and
              (address(23 downto 20) = "1000")
           else '0';
```

Then change the port map to use `eth_sel` instead of raw `iosn`:

```
vhdl

eth : eth_mac
  port map (
    rst => rst,
    clk_25 => clk,
    clk_50 => clk3,
    eth_in => ethi,
    eth_out => etho,
    sram_addr => sram_address,
    sram_data => sram_dat,
    sram_ce1 => sram_ce,
    sram_wen => sram_we,
    sram_oen => sram_oe,
    proc_addr => address,
    proc_data => data,
    proc_ce1 => eth_sel,      -- changed from iosn
    proc_wen => writen,
```

```
    proc_oen => oen
  );
```

This alone fixes bug 2, but **bug 1 (the bus conflict) is the one actually causing your crash**, because even with a correct chip-select, `eth_mac`'s internal data pad still needs to know **not to drive data** unless it is genuinely selected and being read from. Open `eth_mac_p.vhd`'s pad instantiation again:

```
vhdl

proc_data_pad : for i in 0 to DATAWIDTH-1 generate
  data_pad : pc3b04
  port map (PAD => proc_data_p(i), I => mac_proc_rdata(i),
            CIN => mac_proc_wdata(i), OEN => proc_paden);
end generate;

proc_paden <= mac_proc_oen and mac_proc_ce1;
```

This is correct **inside** `eth_mac_p` — the pad only drives the bus (`OEN` active) when both `proc_oen` (read strobe) and `proc_ce1` (chip select) are active together. So once you fix `proc_ce1` to be the real decoded `eth_sel` instead of blanket `iosn`, the pad will correctly stay tristated (not driving `data`) for every access that isn't genuinely addressed to GRETH — which resolves the bus conflict, because `eth_mac` will simply stop driving `data` except during a real, address-decoded GRETH read.

Why you're not actually using `eth_mac_p` yet — and that's fine for now

You confirmed your testbench currently instantiates `eth_mac` (document 2, the unpadded version), not `eth_mac_p`. That's actually the right choice for simulation — `eth_mac_p` exists to add real I/O pads (`pc3d21` / `pc3o02` / `pc3b04`) for an actual FPGA pinout, which you don't need inside a closed testbench where everything stays on internal signals. You can keep using `eth_mac` directly; just fix the `proc_ce1`/decode issue above, since `eth_mac`'s own internal `bridge_top` instance already does the necessary `inout`-vs-split conversion correctly (document 2's architecture maps `bridge_top0`'s real ahb/apb `inout`-free record types straight to `greth0`, and only the very outer testbench-facing port needs the true tristate `data` / `proc_data` style port, which `eth_mac`'s entity already declares correctly as `inout`).

Summary of the actual fix you need to apply

What's wrong	Fix
<code>proc_ce1 => iosn</code> selects <code>eth_mac</code> for ALL IO accesses	Add an address-decoded <code>eth_sel</code> signal, gate on a unique address range, use <code>eth_sel</code> instead of <code>iosn</code>

What's wrong**Fix**

`data` bus has multiple unconditional drivers fighting

This is automatically resolved once `eth_sel` is correctly decoded — `eth_mac`'s pad only drives `data` when truly selected

`eth_mac.proc_data => data` directly

This wiring is fine and intentional — no change needed once selection is correct

`sram1` private SRAM signals (`sram_address` / `sram_dat` /etc.)

Structurally fine; just double check `sramdepth` / `sramfile` aren't reusing LEON3's own SDRAM depth/SREC by mistake — give it its own depth matching `eth_mac`'s internal `ABITS` generic (28) and ideally its own blank `.srec` (or none) since GRETH doesn't need to boot from it

The one thing I still need from you to give you the exact bit range for `eth_sel`: what address did you intend GRETH to live at on this raw LEON3 memory bus (the `address` signal is the word address LEON3 drives directly, not a byte address through an AHB/APB hierarchy anymore, since you're bypassing the SoC's normal AMBA plug&play decode entirely by going straight from `leon3mp`'s raw `address` / `data` / `iosn` pins into `bridge_top`)?

in my system test i changed the base address to 0x20000d00 and then doing it? i don't get why the iosn is a problem since the grtestmod test that was used before directly used iosn.. INSIDE MY ETH_MAC.VHD HAS SRAM_RDATA AND SRAM_WDATA AND PROC_RDATA AND PROC_WDATA .. I NEED TO KNOW HOW TO MAKE THE SIGNALS SRAM_RDATA AND SRAM-WDATA AND PROC_WDATA AND PROC_RDATA COMING FROM BRIDGE _TO TO BE MADE INTO SRAM_DATA AND PROC_DATA..THEN ONLY I WILL BE ABLE TO USE ETH_MAC INSTEAD... TELL ME WHAT TO DO FOR BOTH WHEN I USE ETH_MAC.VHD AND ETH_MAC_PVHD (ALSO HOW TO ACTUALLY DO THE SRAM INSTANCE)

Good questions — let me untangle each one carefully, because the `grtestmod` point is actually the key insight that tells us `iosn` alone is fine, and I was wrong to flag it as the bug.

Why `iosn` is NOT actually the problem

You're right to push back. Look at `grtestmod` in the original commented-out line:

```
vhdl

test0 : grtestmod
  port map ( rst, clk, errorn, address(21 downto 2), data,
            iosn, oen, writen, brdyn);
```

`grtestmod` also took raw `iosn` with no extra decode and worked fine in the original design. The reason that's safe is that **in the stock LEON3 demo design, `grtestmod` is the only device living in IO space** — `sram0`/`prom0` decode on `ramsn`/`romsn`, never on `iosn`, so there's no collision. The same logic applies to your `eth_mac` — if `eth_mac` is the *only* thing using `iosn` in your testbench (and `sram0`/`prom0` use `ramsn`/`romsn` as normal), then `iosn` alone is perfectly fine as the chip-select. I was overcomplicating that part — withdrawn.

So the crash is **not** an address-decode problem. It's purely the **signal renaming inside `eth_mac.vhd`** that you're asking about now. Let's fix that properly.

The actual fix: converting `sram_rdata/sram_wdata` and `proc_rdata/proc_wdata` into true `inout` ports

Inside `eth_mac.vhd` (document 2), the entity currently has **4 separate signals**:

```
vhdl

sram_rdata : in  std_logic_vector(31 downto 0);  -- data coming IN from SRAM
sram_wdata : out std_logic_vector(31 downto 0);  -- data going OUT to SRAM
...
proc_wdata : in  std_logic_vector(31 downto 0);  -- data coming IN from process
proc_rdata : out std_logic_vector(31 downto 0);  -- data going OUT to processor
```

To merge each pair into one true `inout` port, you need a **tristate driver** inside `eth_mac.vhd` itself — exactly the same trick `eth_mac_p.vhd` does with its `pc3b04` pads, just written directly in VHDL instead of using a physical pad component (since you're not synthesizing for an FPGA pin here, you're just merging signals inside simulation).

Step 1 — Change the entity port declarations

In `eth_mac.vhd`, replace:

```
vhdl

sram_rdata : in  std_logic_vector(31 downto 0);
sram_addr  : out std_logic_vector(ABITS-1 downto 0);
sram_wdata : out std_logic_vector(31 downto 0);
sram_ce1   : out std_logic;
```



```
sram_wen    : out std_ulogic;
sram_oen    : out std_ulogic;
```

with:

```
vhdl

sram_addr   : out std_logic_vector(ABITS-1 downto 0);
sram_data   : inout std_logic_vector(31 downto 0);
sram_ce1    : out std_logic;
sram_wen    : out std_ulogic;
sram_oen    : out std_ulogic;
```

And replace:

```
vhdl

proc_wdata  : in  std_logic_vector(31 downto 0);
proc_addr   : in  std_logic_vector(ABITS-1 downto 0);
proc_ce1    : in  std_logic;
proc_wen    : in  std_logic;
proc_oen    : in  std_logic;
proc_rdata  : out std_logic_vector(31 downto 0);
```

with:

```
vhdl

proc_addr   : in    std_logic_vector(ABITS-1 downto 0);
proc_data   : inout std_logic_vector(31 downto 0);
proc_ce1    : in    std_logic;
proc_wen    : in    std_logic;
proc_oen    : in    std_logic;
```

Step 2 — Add tristate driving logic in the architecture body

Inside `eth_mac.vhd`'s architecture (where

`mac_mem_rdata` / `mac_mem_wdata` / `mac_proc_rdata` / `mac_proc_wdata` are still declared as internal signals — keep those exactly as-is, since `bridge_top0` still wants them split internally), add this **after** the `bridge_top0` instantiation:

```
vhdl

-- SRAM side: drive sram_data only when bridge_top wants to write to SRAM (sram_
sram_data <= mac_mem_wdata when sram_wen = '0' else (others => 'Z');
mac_mem_rdata <= sram_data;
```

```
-- Processor side: drive proc_data only when bridge_top wants to return read data
proc_data <= mac_proc_rdata when (proc_ce1 = '0' and proc_oen = '0') else (other
mac_proc_wdata <= proc_data;
```

Note on polarity: check your `sram_wen` / `proc_oen` / `proc_ce1` active levels — in the GRLIB convention these are typically active-low (`'0'` = asserted), which is what I used above, matching `sram_wen` / `oen` being `std_ulogic` driven low-active elsewhere in your files (e.g. `sram_we` / `sram_oe` driving the generic `sram` component the same way `sram0` / `sram1` do in `testbench.vhd`). Confirm against your `sram` component's own polarity assumption before locking this in — if `sram.vhd`'s `we` / `oe` are active-high instead, flip the comparisons to `= '1'`.

This is the **same tristate-merge pattern** `eth_mac_p.vhd` already does via `pc3b04`, just written as plain VHDL signal assignments instead of instantiating a pad component — appropriate since you're not generating a real FPGA bitstream here, only running a behavioral simulation.

Step 3 — Update `eth_mac`'s component declaration in `testbench.vhd` to match

Your testbench's `component eth_mac` declaration must now exactly mirror the new entity:

```
vhdl

component eth_mac
  generic (ABITS : integer := 28);
  port (
    rst : in std_logic;
    clk_25, clk_50 : in std_logic;
    eth_in : in eth_in_type;
    eth_out : out eth_out_type;
    sram_addr : out std_logic_vector(ABITS-1 downto 0);
    sram_data : inout std_logic_vector(31 downto 0);
    sram_ce1 : out std_logic;
    sram_wen : out std_ulogic;
    sram_oen : out std_ulogic;
    proc_addr : in std_logic_vector(ABITS-1 downto 0);
    proc_data : inout std_logic_vector(31 downto 0);
    proc_ce1 : in std_logic;
    proc_wen : in std_logic;
    proc_oen : in std_logic;
```

```
);
end component;
```

This now matches what you already had in the testbench (document 7, page 3) — so once you make the *entity* match this, the mismatch disappears completely and the port map you already wrote (`sram_data => sram_dat`, `proc_data => data`, etc.) becomes valid and correctly typed.

For `eth_mac_p.vhd` — you don't need to touch it at all for this

`eth_mac_p.vhd` already has true `inout` ports (`sram_data_p`, `proc_data_p`) because it uses the physical `pc3b04` tristate pad component, which inherently does this merge for you (`PAD` is the single bidirectional wire; `I` / `CIN` / `OEN` are the split signals feeding/reading it). You only needed the manual tristate-merge fix for `eth_mac.vhd` specifically, since that's the unpadded version where the split signals were left exposed at the top level instead of being merged through a pad. Once `eth_mac.vhd` is fixed as above, both versions present clean `inout` ports at their boundary and either one can be instantiated in your testbench with the same `sram_data` / `proc_data` style port map.

Step 4 — The dedicated SRAM instance for `eth_mac` (your `sram1` block)

Your current `sram1` reuses `sramdepth` (19 bits, sized for LEON3's main SDRAM) and `sramfile` (`"systest.srec"`, LEON3's program binary) — both wrong for GRETH's private buffer memory. Fix it like this:

```
vhdl

-- Add a new constant for eth_mac's private SRAM file (can be empty/blank, GRETH
constant ethsramfile : string := "ethsram.srec"; -- create an empty file with t

signal sram_address : std_logic_vector(27 downto 0); -- match ABITS=28 used by
signal sram_dat      : std_logic_vector(31 downto 0);
signal sram_ce, sram_we, sram_oe : std_ulogic;

sram1 : for i in 0 to (sramwidth/8)-1 generate
    sr1 : sram generic map (index => i, abits => 28, tacc => 10, fname => ethsram
        port map (sram_address(27 downto 2), sram_dat(31-i*8 downto 24-i*8),
            sram_ce, sram_we, sram_oe);
end generate;
```

Key changes from what you have: `abits => 28` (matching `eth_mac`'s `ABITS` generic, not `sramdepth`), and a **separate, dedicated** `.srec` filename so it isn't trying to load

your CPU program binary into GRETH's frame buffer space. You can create an empty placeholder file:

Create an empty placeholder srec file for the eth_mac private SRAM >

An empty `.srec` is fine — the generic `sram` model in GRLIB initializes the rest of memory to zero/undefined when the file has nothing to load, and GRETH will write its own descriptors and frame buffers into this SRAM at runtime anyway, so it doesn't need preloaded contents.

Summary checklist

Item	Action
<code>eth_mac.vhd</code> entity ports	Merge <code>sram_rdata</code> / <code>sram_wdata</code> → <code>sram_data : inout</code> ; merge <code>proc_wdata</code> / <code>proc_rdata</code> → <code>proc_data : inout</code>
<code>eth_mac.vhd</code> architecture	Add the two tristate-merge assignment pairs shown in Step 2, check active-low vs active-high polarity against your <code>sram.vhd</code>
testbench.vhd component declaration	Update to match the new entity exactly (already matches what you wrote, once the entity catches up)
testbench.vhd port map (<code>eth :</code> <code>eth_mac port map(...)</code>)	No change needed — what you have is already correct once the entity is fixed
<code>proc_ce1 => iosn</code>	Keep as-is — confirmed fine since <code>eth_mac</code> is the only IO-space device
<code>sram1</code> private SRAM block	Change <code>abits</code> from <code>sramdepth</code> to <code>28</code> , change <code>fname</code> from <code>sramfile</code> to a dedicated empty <code>ethsramfile</code>
<code>eth_mac_p.vhd</code>	No changes required — already correctly merges via <code>pc3b04</code> pads

Once you apply the `eth_mac.vhd` entity/architecture fix and the `sram1` depth/file fix, the `inout` typing error that was likely failing elaboration (or causing the bus-driver conflict producing `x` at time 0) should be resolved, and the crash before any printf output should clear up. Re-run and let me know what the transcript shows next.

Add the two tristate-merge assignment pairs shown in Step 2, I THINK U MIXED eth_mac.vhd and eth_mac_p.vhd ...BECAUSE eth_mac.vhd does

not have the `mac_mem_wdata` and stuff. (ALSO `sram_wen` is an out port)
it will give error

You're completely right on both counts — let me correct this properly.

Catching my error

You're right that `mac_mem_wdata` / `mac_mem_rdata` / `mac_proc_wdata` / `mac_proc_rdata` are signals declared inside `eth_mac_p.vhd`'s architecture (document 3) — they don't exist in `eth_mac.vhd` (document 2) at all. In `eth_mac.vhd`, the architecture only declares `apbi` / `apbo` / `ahbmi` / `ahbmo` and directly wires `greth0` and `bridge_top0` together — there's no intermediate `mac_*` signal layer to tap into.

You're also right that `sram_wen` is an **out** port on `eth_mac` (driven by `bridge_top0` inside it, going out to the external SRAM) — so I cannot use it as a condition to drive `sram_data` from outside-in logic the way I wrote it; that's backwards. The actual read/write strobe needs to come from **inside** `bridge_top0`, not be read back from its own output port as a feedback condition (that would be circular and also just structurally wrong, since `sram_wen` / `sram_oen` are what `eth_mac` is generating, not consuming).

The correct fix for `eth_mac.vhd`

Since there's no `mac_*` intermediate signal in `eth_mac.vhd`, the tristate merge has to happen using `bridge_top0`'s own ports directly. Looking at `bridge_top`'s entity (document 1), `bridge_top0` is instantiated inside `eth_mac.vhd` with these memory-side ports:

```
vhdl

-- From bridge_top entity (document 1, page 3)
sram_rdata : in  std_logic_vector(31 downto 0);  -- INTO bridge_top, from SRAM
sram_addr  : out std_logic_vector(ABITS-1 downto 0);
sram_wdata : out std_logic_vector(31 downto 0);  -- OUT of bridge_top, to SRAM
sram_ce1   : out std_logic;
sram_wen   : out std_ulogic;
sram_oen   : out std_ulogic;

proc_wdata : in  std_logic_vector(31 downto 0);  -- INTO bridge_top, from proce
proc_addr  : in  std_logic_vector(ABITS-1 downto 0);
proc_ce1   : in  std_logic;
proc_wen   : in  std_logic;
proc_oen   : in  std_logic;
proc_rdata : out std_logic_vector(31 downto 0);  -- OUT of bridge_top, to proce
```

So inside `eth_mac.vhd`'s architecture, you need **internal signals** to connect to `bridge_top0`'s split ports, then merge those into the new `inout` entity ports. Here's the complete corrected version:

Step 1 — Declare internal split signals in `eth_mac.vhd`'s architecture

```
vhdl

signal bt_sram_rdata : std_logic_vector(31 downto 0);
signal bt_sram_wdata : std_logic_vector(31 downto 0);
signal bt_sram_wen   : std_ulogic;
signal bt_sram_oen   : std_ulogic;

signal bt_proc_wdata : std_logic_vector(31 downto 0);
signal bt_proc_rdata : std_logic_vector(31 downto 0);
```

Step 2 — Wire `bridge_top0`'s port map to these internal signals instead of the old entity ports

Find the existing `bridge_top0` instantiation inside `eth_mac.vhd` and change its memory-side port map connections:

```
vhdl

bridge_top0 : bridge_top
  generic map (ABITS)
  port map (
    clk => clk, rst => rst,
    -- AHB/APB side unchanged --
    ...
    -- Memory side: connect to internal split signals, NOT directly to entity po
    sram_rdata => bt_sram_rdata,
    sram_addr  => sram_addr,      -- entity port, unchanged (out, no conflict)
    sram_wdata => bt_sram_wdata,
    sram_ce1   => sram_ce1,      -- entity port, unchanged (out, no conflict)
    sram_wen   => bt_sram_wen,
    sram_oen   => bt_sram_oen,

    proc_wdata => bt_proc_wdata,
    proc_addr  => proc_addr,      -- entity port, unchanged (in, no conflict)
    proc_ce1   => proc_ce1,      -- entity port, unchanged (in, no conflict)
    proc_wen   => proc_wen,      -- entity port, unchanged (in, no conflict)
    proc_oen   => proc_oen,      -- entity port, unchanged (in, no conflict)
    proc_rdata => bt_proc_rdata
  );
```

Note `sram_addr`, `sram_ce1`, `proc_addr`, `proc_ce1`, `proc_wen`, `proc_oen` stay wired straight to the entity ports as before — those are single-direction signals with no merging needed. Only the **data lines** and the **two strobes used as tristate conditions** (`sram_wen`, `proc_oen` / `proc_ce1`) need the internal tap.

Step 3 — Add the tristate merge logic, now driven from the correct internal signals

```
vhdl

-- SRAM side: bridge_top0 asserts bt_sram_wen low when it wants to WRITE to SRAM
sram_data      <= bt_sram_wdata when bt_sram_wen = '0' else (others => 'Z');
bt_sram_rdata <= sram_data;

-- Processor side: bridge_top0 wants to drive proc_data when chip-selected AND r
proc_data      <= bt_proc_rdata when (proc_ce1 = '0' and proc_oen = '0') else (ot
bt_proc_wdata <= proc_data;
```

This now correctly uses **internal signals tapped from `bridge_top0`'s own outputs** (`bt_sram_wen`) as the tristate condition, rather than trying to read back `eth_mac`'s own output port — and it doesn't reference anything that only exists in `eth_mac_p.vhd`.

Confirm polarity before locking this in

Check `bridge_top.vhd`'s internal logic (inside the AHB/APB bridge components, not shown in the pages I have) for whether `sram_wen` / `proc_oen` / `proc_ce1` are driven active-low or active-high. Everything in GRLIB convention (and consistent with `oen` / `writen` naming elsewhere in your testbench, e.g. LEON3's own `oen` / `writen` signals) suggests active-low, which is what I used (`= '0'`). If a quick simulation shows `sram_data` / `proc_data` never get driven (stuck at `Z` / `X`), flip these comparisons to `= '1'` first before suspecting anything else.

One thing to double check on your end

I don't have visibility into the internal logic of the `AHB bridge` / `APB bridge` sub-blocks referenced inside `bridge_top.vhd` (only the top-level entity/port map was in the PDF). If `bridge_top`'s memory-side `sram_wen` / `proc_oen` happen to be registered (delayed by a clock cycle) rather than combinational, the tristate condition in Step 3 still works correctly since it's just combinational logic following whatever `bt_sram_wen` / `proc_oen` actually do — no change needed there either way.

So what exactly do u think is the error for the simulation stop? How do i stop that? Because it probably wasn't the issue with the inout ports(

because even when i used `eth_mac_p` the simulation stop happened too)
 ..but rather with something during the port mapping.. tell me how to port
 map crtclly (the code itself for the testbench)

You're right — if the crash happens identically with **both** `eth_mac` and `eth_mac_p`, the `inout`-merge issue can't be the cause, since `eth_mac_p` already has correct native `inout` ports via `pc3b04` pads with zero hand-editing needed. The bug has to be something common to both paths: the **port map itself**, specifically what's connected to `proc_addr`, `proc_data`, `proc_ce1`, `proc_wen`, `proc_oen` from the LEON3 side.

Let me trace this very carefully against the actual `leon3mp` port list in `testbench.vhd` (document 7, page 2) since that's the one piece I haven't scrutinized closely enough yet.

The real candidate: `address` width mismatch

Look at `leon3mp`'s actual port declaration:

```
vhdl
address : out std_logic_vector(27 downto 0);
data     : inout std_logic_vector(31 downto 0);
```

`address` is **28 bits** (`27 downto 0`), word-aligned addressing already baked in at the LEON3 level for a 28-bit physical address space — this is a `mem_in_type`/raw-bus style signal, **not** a byte address. Now look at `eth_mac`'s entity:

```
vhdl
proc_addr : in std_logic_vector(ABITS-1 downto 0);    -- ABITS default = 28
```

That matches — 28 bits both sides, so width isn't the mismatch. Good, ruled out.

The real candidate: `iosn`/`oen`/`writen` are single bits, but timing/protocol may not match what `bridge_top`'s `proc_*` side expects

Here's the structural issue I think we've been missing. `bridge_top`'s `proc_*` ports were designed to be driven by something that already speaks a **simple synchronous memory protocol** matching exactly what GRLIB's generic `sram`/`mem_apb_bridge`/`ahb_mem_bridge` components expect: `proc_ce1`, `proc_wen`, `proc_oen` as direct strobes sampled on `clk`, with `proc_addr`/`proc_wdata` valid for the same cycle.

But `address`/`data`/`iosn`/`writen`/`oen` coming straight off `leon3mp`'s top-level pins are **LEON3's own external memory bus**, which has its own specific timing relationship (the cycle in which `oen`/`writen`/`iosn` are valid relative to `address`,

whether there's a wait-state/`brdyn` requirement, etc.) — this is the same bus that `sram0`/`prom0`/`grtestmod` connect to, and they work because the generic `sram` component (and presumably `grtestmod`) were written specifically to match LEON3's external bus timing.

`bridge_top`'s `proc_*` ports were almost certainly not written to match LEON3's raw external bus timing — they were written to match `eth_mac`'s own internal expectation of a generic memory-mapped peripheral bus, which is a different (likely much simpler, no-wait-state) protocol than LEON3MP's external pin-level bus.

This is the kind of mismatch that produces exactly your symptom: not a compile error, not even necessarily wrong logic values — but a **protocol/timing mismatch** that causes LEON3 to either never see `brdyn` come back correctly, or to see garbage `data` because it sampled before `eth_mac` had valid `proc_rdata` ready, which then corrupts an early fetch/access and trips `iuerr` almost immediately.

I need one specific piece of information to confirm this rather than keep guessing

Q: Does your current `eth_mac` port map in `testbench.vhd` connect anything to `brdyn` (the wait-state/ready signal `leon3mp` expects back from memory), or is `brdyn` only being driven by `sram0`/`prom0`'s logic and `eth_mac` is just ignored for that signal?

A: Not sure / need to check

Q: Do you have the actual VHDL body (architecture, not just the entity/port list) for `bridge_top`'s internal 'APB bridge' and 'AHB bridge' sub-components? I need to see whether they expect a wait-state protocol matching LEON3's raw bus, or assume single-cycle/combinational response.

A: I do have it, let me check first

Good — please go check both of those, since they're the two things that will actually tell us what's wrong rather than me continuing to guess.

While you're checking, here's exactly what to look for in each, so you know what matters:

For `brdyn`

Look at `testbench.vhd`'s `data <= buskeep(data) after 5 ns;` line and the `leon3mp` port map — find what's currently wired to `brdyn`. In the original working design (before you touched anything), `brdyn` is probably tied to a constant (`'1'`, meaning

"always ready, no wait states") since `sram0` / `prom0`'s generic `sram` model responds combinatorially with no extra wait state needed. If `bridge_top`'s `proc_*` side needs even one extra cycle to fetch from GRETH (going through APB or AHB internally takes multiple clock cycles in real hardware), but `brdyn` is hardwired to `'1'`, LEON3 will think data is ready a cycle (or many cycles) before `bridge_top` has actually produced valid `proc_rdata`. That mismatch alone would explain garbage being read immediately at boot.

For the `bridge_top` internals

Specifically check:

- Does the **APB bridge** sub-block assert `apbo` / wait for `pready` -equivalent before returning data, or does it just combinatorially pass through in the same cycle?
- Does the **AHB bridge** sub-block use `hready` / multi-cycle AHB transaction handshaking (which inherently takes 2+ cycles per real AHB protocol), and if so, does that latency get reflected anywhere back to the `proc_*` side, or does `proc_rdata` / the implicit ready signal just appear whenever it appears with nothing telling LEON3 to wait?

If the AHB/APB bridge logic takes multiple cycles internally (very likely, since real APB/AHB transactions are never single-cycle) but the `proc_*` interface offers **no way to signal "not ready yet"** back to LEON3 (no `proc_brdyn` or equivalent port exists in the entity at all, based on what's in the PDF), that's the structural gap: `bridge_top` was designed assuming whatever drives its `proc_*` side already handles wait-states externally — but `leon3mp`'s raw bus expects the **memory device itself** to either be single-cycle or properly extend `brdyn` until ready, and `bridge_top` doesn't expose a `brdyn`-style port to do that.

That would explain why it fails identically with both `eth_mac` and `eth_mac_p` — the timing problem is entirely inside `bridge_top`, common to both wrappers, and has nothing to do with the `inout` vs split-signal question we were chasing before.

Paste back what you find on `brdyn` and the `bridge_top` architecture body and I'll confirm exactly where the fix needs to go — whether that's adding wait-state logic in `bridge_top`, or stretching LEON3's access via `brdyn` from the testbench side until `bridge_top` signals real completion.

I only have the old version of APB and AHB...where the logic was different..so consider that I don't have it . And also i am not sure about brdyn...so yea..Tell me how to Port map the eth_mac_p inside the testbench

Fair enough — without the actual `bridge_top` internals or confirmed `brdyn` wiring, let's stop guessing at internal timing and focus on what we can control directly: giving you a complete, correct port map for `eth_mac_p` inside `testbench.vhd`, built strictly from the entity ports we do have (document 3), wired the way LEON3's raw bus and a dedicated SRAM actually need to be connected. I'll also flag the `brdyn` question explicitly as something you must resolve, since I can't see `bridge_top`'s internals to confirm it.

Step 1 — Signals you need declared in `testbench.vhd`

Add these alongside your existing signal declarations (most you likely already have from your `eth_mac` attempt — reuse them):

```
vhdl

-- Clock/reset already exist: clk, clk3, rst -- reuse these, do not redeclare

-- Ethernet PHY interface (already exists: ethi, etho via eth_in_type/eth_out_ty
-- reuse ethi/etho exactly as before -- eth_mac_p has the SAME eth_in/eth_out po

-- Dedicated SRAM signals for eth_mac_p's private SRAM
signal eth_sram_addr  : std_logic_vector(27 downto 0);
signal eth_sram_data  : std_logic_vector(31 downto 0);
signal eth_sram_ce1   : std_logic;
signal eth_sram_wen   : std_ulogic;
signal eth_sram_oen   : std_ulogic;

-- Processor side -- these connect straight to LEON3's raw bus
-- reuse address, data, iosn, writen, oen -- already exist from leon3mp port map
```

Step 2 — The component declaration for `eth_mac_p`

This must mirror document 3's entity exactly:

```
vhdl

component eth_mac_p
  generic (
    ABITS      : integer := 28;
    SRAM_BANKS: integer := 4;
    DATAWIDTH : integer := 32
  );
  port (
    rst_p, clk_50_p, clk_25_p : in std_logic;
```

```

-- Ethernet PHY
emdio_p  : inout std_logic;
etx_clk_p : in  std_ulogic;
erx_clk_p : in  std_ulogic;
erxd_p    : in  std_logic_vector(3 downto 0);
erx_dv_p  : in  std_ulogic;
erx_er_p  : in  std_ulogic;
erx_col_p : in  std_ulogic;
erx_crs_p : in  std_ulogic;
emdint_p  : in  std_ulogic;
etxd_p    : out std_logic_vector(3 downto 0);
etx_en_p  : out std_ulogic;
etx_er_p  : out std_ulogic;
emdc_p    : out std_ulogic;

-- SRAM
sram_addr_p : out std_logic_vector(ABITS-1 downto 0);
sram_data_p : inout std_logic_vector(31 downto 0);
sram_ce1_p  : out std_logic;
sram_wen_p  : out std_ulogic;
sram_oen_p  : out std_ulogic;

-- Processor
proc_addr_p : in    std_logic_vector(ABITS-1 downto 0);
proc_data_p : inout std_logic_vector(31 downto 0);
proc_ce1_p  : in    std_logic;
proc_wen_p  : in    std_logic;
proc_oen_p  : in    std_logic
);
end component;
```

Step 3 — The actual instantiation/port map

Replace whatever `eth : eth_mac port map(...)` block you currently have with this:

```

vhdl

eth : eth_mac_p
  generic map (
    ABITS      => 28,
    SRAM_BANKS => 4,
    DATAWIDTH => 32
  )
  port map (
    rst_p      => rst,
    clk_50_p   => clk3,    -- matches your existing clk3 = 50MHz-style clock use
```

```

clk_25_p => clk,          -- matches your existing clk = system clock

-- Ethernet PHY -- wire to the SAME signals the phy0 instance already uses
emdio_p  => emdio,
etx_clk_p => etx_clk,
erx_clk_p => erx_clk,
erxd_p   => erxd,
erx_dv_p => erx_dv,
erx_er_p => erx_er,
erx_col_p => erx_col,
erx_crs_p => erx_crs,
emdint_p => emdint,
etxd_p   => etxd,
etx_en_p => etx_en,
etx_er_p => etx_er,
emdc_p   => emdc,

-- Dedicated SRAM (private to eth_mac_p, NOT leon3's main sram0)
sram_addr_p => eth_sram_addr,
sram_data_p => eth_sram_data,
sram_ce1_p  => eth_sram_ce1,
sram_wen_p  => eth_sram_wen,
sram_oen_p  => eth_sram_oen,

-- Processor side -- LEON3's raw external memory bus
proc_addr_p => address(27 downto 0),
proc_data_p => data,
proc_ce1_p  => iosn,
proc_wen_p  => writen,
proc_oen_p  => oen
);

```

Important note on the PHY signals: I wired `emdio_p` / `etx_clk_p` / etc. to the same bare signals (`emdio`, `etx_clk`, `erx_clk` ...) that your testbench already declares and uses for the existing `phy0` instance (document 7, page 6: `p0: phy generic map(...) port map(rst, emdio, etx_clk, erx_clk, erxd, erx_dv, erx_er, erx_col, erx_crs, etxd, etx_en, etx_er, emdc, eth_macclk)`). Since you're now routing Ethernet through `eth_mac_p` instead of the original `eth0` / `greth` direct instantiation, **you need to make sure the same `phy0` instance is still driving these signals to `eth_mac_p` instead of to wherever it was connected before** — don't duplicate a second PHY instance, just **repoint the existing one's connections to `eth_mac_p`'s ports.**

Step 4 — The dedicated SRAM instance for eth_mac_p's private memory

```

vhdl

constant ethsramfile : string := "ethsram.srec";

eth_sram1 : for i in 0 to (sramwidth/8)-1 generate
    eth_sr1 : sram generic map (index => i, abits => 28, tacc => 10, fname => eth
        port map (eth_sram_addr(27 downto 2), eth_sram_data(31-i*8 downto 24-i*8
            eth_sram_ce1, eth_sram_wen, eth_sram_oen);
end generate;

```

Step 5 — Remove the old `grtestmod` and old `eth_mac` instantiations completely

Make sure you don't have **both** the old commented-out `grtestmod`/direct-`greth` wiring **and** this new `eth_mac_p` block trying to drive the same `ethi`/`etho`/PHY signals simultaneously — that alone would cause exactly this kind of silent corruption regardless of the inout question. Confirm only one Ethernet-related component instantiation is active in the file.

On `brdyn` — this is the part you must verify before re-running

You said you're not sure about `brdyn`. Find this signal in your testbench and check what drives it currently:

```

vhdl

-- search for "brdyn" in your testbench.vhd

```

If it's tied to a constant `'1'` (always ready) anywhere, and `bridge_top`'s internal logic genuinely needs more than one cycle to complete an APB or AHB transaction internally, LEON3 will read garbage before `eth_mac_p` has valid data ready — this remains the most likely root cause of an immediate `iuerr`, independent of the port map itself. Since I don't have the `bridge_top` internals, I can't confirm the exact fix here, but if `brdyn` is hardwired high, that's the next thing to investigate if this port map alone doesn't resolve the crash.

I have sent the testbench to u..can't u check your self? Also if I am to do port mapping for `eth_mac` give me that as well

You're right — let me actually go back and check the testbench file precisely instead of asking again.

Find the uploaded testbench and related files >

I see — there's an actual `testbench.vhd` file on disk in addition to the PDF I was reading earlier. Let me read that directly.

View the actual testbench.vhd source file >

Now I have the **actual** file, and this completely changes the diagnosis. This is critically important — this version has **no** `eth_mac` / `eth_mac_p` **instantiation at all**. This is the original, unmodified GRLIB demo testbench. Your "modified testbench" with the `eth : eth_mac port map(...)` block shown in the earlier PDF screenshots is a **different file** than what's actually on disk here.

Let me also pull up `bridge_top.pdf` and `eth_mac_vhd.pdf` / `eth_mac_p_vhd.pdf` again carefully to re-confirm exact signal names before writing the port maps, since precision matters a lot here.

List all uploaded files to check for any other testbench versions >

Confirmed — this is a stale `testbench.vhd` from May 27, completely separate from the modified version you described in your messages (the one with `eth : eth_mac port map(...)`, `sram1` block, etc.), and also separate from `testbench_vhd.pdf` which is more recent (Jun 17, the one with the "ADDED ON 15/06/26" comments showing the actual `eth_mac` instantiation). I was working from the PDF correctly for the diagnostic conversation, but **this `.vhd` file you just pointed me to is not your current testbench at all** — it's the original baseline before you started modifying it.

I also notice there's `leon3mp.vhd` and `phy.vhd` on disk now, which I haven't checked yet. Let me look at those since they directly affect the correct port map.

Check how leon3mp.vhd instantiates GRETH currently >

This is the critical discovery. `leon3mp.vhd` **already has GRETH instantiated internally** (line 566, `eth0 : if CFG_GRETH = 1 generate`) — meaning the **original on-chip GRETH is still there, fully connected to LEON3's internal AHB/APB bus**, completely separate from any `eth_mac` / `eth_mac_p` external module you're trying to add via the raw `address` / `data` / `iosn` pins.

View the eth0 GRETH instantiation in leon3mp.vhd ✓

 View the eth0 GRETH instantiation in leon3mp.vhd

 Done

This confirms it precisely. GRETH (`grethm` , `e1`) is still instantiated inside `leon3mp.vhd` , fully wired to the real internal `ahbmi` / `ahbmo` / `apbi` / `apbo` signals with `pindex=>13` , `paddr=>13` — the exact GRETH that your original `greth_test.c` (with `GRETH_BASE 0x80000D00`) was successfully testing against, on the **real internal AMBA bus**, completely independent of anything you're doing at the testbench level with raw `address` / `data` / `iosn` .

This means two GRETH-capable paths exist **simultaneously** in your design right now:

1. **Internal** `eth0` / `grethm` inside `leon3mp.vhd` — on the real AHB/APB bus, address `0x80000D00` , this is what worked perfectly when all 5 tests passed.
2. **Your new external** `eth_mac` / `eth_mac_p` — sitting outside `leon3mp` on the raw `address` / `data` / `iosn` pins at the top testbench level, address `0x20000D00` per your last message.

I now need to know exactly what you intended for `CFG_GRETH` and the internal `eth0` block, because this directly determines what's colliding.

Q: In your config.vhd, is CFG_GRETH still set to 1 (meaning the internal eth0/grethm block in leon3mp.vhd is still active and instantiated alongside your new external eth_mac_p), or did you set it to 0 to remove the internal GRETH before adding the external one?

A: Yes is it still there and i did not remove it but rather in the testbench I have modified I have made those signals that are needed as open and grounded them

Show more

This explains everything. Now I can see the actual bug clearly, and it's not in `eth_mac` / `eth_mac_p` or the inout ports at all — **it's in how `leon3mp`'s port map was modified.**

The bug, precisely

`leon3mp.vhd`'s internal `eth0` / `grethm` block is **still active** (`CFG_GRETH = 1`), which means inside `leon3mp` , this is still happening exactly as before:

```
vhdl

ethpads : if (CFG_GRETH = 1) generate
    emdio_pad : iopad ... port map (emdio, etho.mdio_o, etho.mdio_oe, ethi.mdio_i)
```

```
etxc_pad : clkpad ... port map (etx_clk, ethi.tx_clk);
erxc_pad : clkpad ... port map (erx_clk, ethi.rx_clk);
erxd_pad : inpadv ... port map (erxd, ethi.rxd(3 downto 0));
... etc
```

These are **top-level pins of leon3mp itself** (`emdio`, `etx_clk`, `erx_clk`, `erxd`, etc.) — they are still declared in `leon3mp`'s entity and still being driven internally by the **real, still-active internal GRETH** (`e1 : grethm`).

You said you "grounded/opened" these in the **testbench's** port map to `leon3mp` — but that doesn't disconnect the internal GRETH from them. `leon3mp`'s internal logic still tries to drive `etxd`, `etx_en`, `emdc`, etc. **out** through its own pins regardless of what the testbench connects those pins to externally. If you ground/open them at the testbench level, you're just dangling the internal GRETH's outputs into nothing (which is harmless on its own) — but the real issue is **what's happening to the internal GRETH's APB/AHB bus access**, because that's not exposed as a top-level pin at all — it's purely internal to `leon3mp`, on `apbi` / `apbo` / `ahbmi` / `ahbmo`.

The actual structural conflict

Your external `eth_mac_p` sits on the **raw external bus** (`address` / `data` / `iosn` / `writen` / `oen` — `leon3mp`'s actual top-level memory bus pins), trying to act as a memory-mapped peripheral at `0x20000D00`. But LEON3's **CPU itself**, internally, is still also generating APB transactions to the **internal GRETH** at `pindex=>13, paddr=>13` (which decodes to address `0x800000D0`-range internally, **completely independent of what's on the external `address` / `data` pins**) every single time your C code's `RD()/WR()` macros touch `GRETH_BASE`.

So here's the real question that determines everything:

Your C test code (`greth_test.c`) still has `#define GRETH_BASE 0x80000D00`, which is the address of the **INTERNAL grethm core, not the new `0x20000D00` external one** you mentioned for `eth_mac_p`. If that's still true, your C code is talking to the **still-functional internal GRETH**, and your new `eth_mac_p` external module is sitting there completely unused, electrically isolated from anything the CPU is actually addressing — which means it shouldn't be causing a crash by itself...

unless the crash is coming from something LEON3 does at boot/reset that touches the external bus (`address` / `data` / `iosn`) for an unrelated reason (e.g., PROM fetch, or AMBA plug&play auto-detection sweeping the IO area), and your ungrounded/floating `eth_mac_p` is now reacting to that and corrupting `data` right when LEON3 fetches its first instruction from PROM — which lives on the very same `data` bus.

Q: In the version of greth_test.c you are CURRENTLY compiling and running right now, is GRETH_BASE set to 0x20000D00 (targeting your new external eth_mac_p) or still 0x80000D00 (targeting the original internal grethm)?

A: Yes, I changed greth_test.c's GRETH_BASE to 0x20000D00 to target the new external module

Q: When you 'grounded the old ethernet signals' in leon3mp's port map inside your modified testbench, did you ground/open ONLY the external PHY-facing pins (emdio, etx_clk, erx_clk, erxd, etx_en, emdc etc), or did you also try to touch/disable the internal apbi/apbo/ahbmi/ahbmo signals that grethm uses internally (which are NOT top-level leon3mp ports and can't be touched from the testbench anyway)?

A: Not sure but I think only those..u can check my testbench file i gave

Good — let me actually check the file precisely instead of guessing further, since I have it on disk and should verify rather than speculate.

Locate the leon3mp instantiation in the current testbench file on disk >

This confirms the file on disk is unmodified — `cpu : leon3mp` here still uses the original `emdio, etx_clk, erx_clk, ...` signals connected normally, not grounded. This is **not** the file you're actually compiling with your `eth_mac_p` changes — that version exists somewhere else (probably only in your editor/local copy, not uploaded here). I need that exact file before I can confirm what you actually grounded.

Q: The testbench.vhd on disk is the unmodified original - it doesn't have your eth_mac_p changes at all. To actually fix your real bug (not guess at it), I need your real modified file. What's easiest for you right now?

A: The testbench I sent on June 17 does have the modified for leon3mp Port map...thought yes it uses eth_mac and not eth_mac_p

You're right — let me actually go pull that exact PDF page back up and read it precisely rather than relying on memory of it.

Re-checking testbench_vhd.pdf page 5 for the actual modified leon3mp port map >

The PDF is just scanned images, not extractable as text — I already have the rendered images from earlier in the conversation (document 7, pages 1-8) and that's the actual content. I went down an unproductive path trying to re-view it as if there were new info to extract; there isn't. Let me work directly from page 6's image, which is the one showing the modified `leon3mp` port map with the grounded ethernet signals.

Ran 2 commands, viewed a file >

Now I have the actual page at full resolution. Let me check page 5 too, since that's where the `leon3mp` instantiation and the grounded signals should be.

View page 5 of testbench PDF - leon3mp instantiation with the grounded ethernet signals >

Now I can read this precisely, and this is the actual bug. Look at the new `cpu :` `leon3mp` port map (positional, ordered exactly to match `leon3mp`'s entity port list one-for-one):

```
vhdl

port map (rst, clk, clk3, pllref, errorn, wdog, address(27 downto 0), data,
  ramsn, ramoen, rwen, oen, writen, read, iosn, bexc, brdyn, romsn,
  sdclk, sdc, sdwen, sdrasn, sdc, sddq,
  dsu, dsu, dsu,
  tx1, rx1, cts1, rts1, tx2, rx2, cts2, rts2, pio,
  open, '0', '0', "0000", '0', '0', '0', '0',
  open, open, open, open, ps2clk, ps2data, vid_clock, vid_blank, vid_sync,
  ...
```

This is a **positional port map**, meaning each value must line up exactly, in order, with `leon3mp`'s entity port list. Counting `leon3mp`'s entity ports (from the original `testbench.vhd` on disk, lines 113-125) in order after `pio`:

```
emdio, etx_clk, erx_clk, erxd, erx_dv, erx_er, erx_col, erx_crs, emdint,
etxd, etx_en, etx_er, emdc
```

That's **13 ports**: `emdio` (inout), `etx_clk` (in), `erx_clk` (in), `erxd` (in, 4-bit vector), `erx_dv` (in), `erx_er` (in), `erx_col` (in), `erx_crs` (in), `emdint` (in), then **outputs**: `etxd` (out, 4-bit), `etx_en` (out), `etx_er` (out), `emdc` (out).

Your replacement supplies exactly **8 values** for the first 8 positions (`open`, `'0'`, `'0'`, `"0000"`, `'0'`, `'0'`, `'0'`, `'0'`) covering `emdio` through `emdint` — that part is fine, those are inputs/inout being driven to ground or left open. **But then it gives 4 more values**: `open`, `open`, `open`, `open` for `etxd`, `etx_en`, `etx_er`, `emdc` — and these are `leon3mp`'s output ports.

This is the bug

You cannot connect `open` to an output port the way you'd ground an input. Using `open` on an `out` port in a positional port map is actually legal VHDL syntax (it just means "don't care about this output, don't connect it to anything") — so this wouldn't cause a compile error. But think about what it does electrically: `leon3mp`'s internal `grethm`/`eth0` (still active, `CFG_GRETH=1`) is still driving real values onto

`etxd`/`etx_en`/`etx_er`/`emdc` internally — connecting them to `open` just means the testbench discards those values, which is harmless by itself.

The actual problem is the 8 `'0' / "0000"` values feeding `leon3mp`'s `in`/`inout` ports — specifically `emdio`. Look at `leon3mp`'s entity: `emdio : inout std_logic`. Your replacement list starts with `open` for `emdio` — but then look at `leon3mp.vhd`'s internal logic for `emdio` (line 580-581 from earlier): `emdio_pad : iopad ... port map (emdio, etho.mdio_o, etho.mdio_oe, ethi.mdio_i)` — this is a **bidirectional pad** that both reads and drives `emdio` depending on `etho.mdio_oe`. If you connect `open` to an `inout` port, **the internal driver still drives an open/dangling net internally**, and more importantly, **the GRETH inside `leon3mp` is still actively running its MDIO state machine and IRQ logic against `ethi`/`etho` signals that now have no real external loopback path** — meaning if your C code's `RD()/WR()` at `0x20000D00` were ever to (even accidentally) trigger an internal access pattern that overlaps with the **still-active internal GRETH's own AHB DMA bus arbitration** (since `grethm` is a real AHB master, hindex assigned, requesting bus grants every cycle it has pending work), you get **two AHB masters now contending unpredictably** — but that alone shouldn't crash before `printf` either.

I need to actually count and match positions exactly rather than reasoning abstractly, since a position-by-position counting error is the single most likely concrete bug in a long positional port map like this.

Precisely count and align entity ports vs the new positional port map >

Found it. There's an off-by-one — your replacement block supplied 8 values (`open`, `'0'`, `'0'`, `"0000"`, `'0'`, `'0'`, `'0'`, `'0'`) for what should cover 9 input-side ports (`emdio` through `emdint` is 9 ports: `emdio`, `etx_clk`, `erx_clk`, `erxd`, `erx_dv`, `erx_er`, `erx_col`, `erx_crs`, `emdint`). You're one item short, which shifts every single port after it by one position.

The consequence: `emdint` (an **input** to `leon3mp`, meant to be tied low) ends up receiving `open` from your second group of values, `etxd` gets the first stray `open`, and critically — `emdc` (an **output**) ends up receiving `ps2clk`, meaning `ps2clk` — a real, externally-driven `inout` signal used by the PS/2 device simulation — is now wired into the position where `leon3mp` expects to receive nothing (it's an output position, so this would actually be a **type/direction mismatch error from VCS at elaboration**, not silently wrong). And everything downstream (`ps2data`, `vid_clock`, etc.) is similarly shifted by one, which would cascade into wrong types being connected to nearly every remaining port — `vid_clock` (an out) getting `ps2data` (an inout), and so on, all the way through `usb_vbus`.

This level of type mismatch should make VCS refuse to elaborate at all with a clear port-type error — which makes me suspect what's actually happening is slightly different from what's printed in that PDF transcription (handwriting/scan artifacts could easily have dropped or merged a comma), but the structural lesson holds

regardless: a positional port map with 8 replacement values where 9 are needed will silently or fatally misalign every subsequent connection, and this is exactly the kind of bug that produces nonsensical behavior (X propagation into `data`/instruction stream) within the first few thousand ns — matching your `iuerr` symptom precisely.

The fix — stop using positional mapping for this block entirely

Positional mapping across 80+ ports is exactly the kind of thing that creates silent miscounts like this. Convert this whole instantiation to **named** association instead, which makes alignment errors structurally impossible (a typo in a name just gives a clear "no such port" error instead of silently shifting everything):

Write the corrected named-association `leon3mp` port map for the testbench ▾

 Write the corrected named-association `leon3mp` port map for the testbench

`leon3mp_portmap_fix.vhd`

 Done

Why I changed `emdio => '0'` to `emdio => open`

I kept your other tie-offs (`'0'` / `"0000"`) for the genuine **input** ports (`etx_clk`, `erx_clk`, `erxd`, `erx_dv`, `erx_er`, `erx_col`, `erx_crs`, `emdint`) — those are fine to ground since `leon3mp` only reads them. But `emdio` is `inout`. Tying an `inout` port to a constant `'0'` from the testbench while `leon3mp`'s internal `iopad` is simultaneously trying to drive it (whenever `etho.mdio_oe` goes active, which it will, since the internal `grethm` MDIO state machine runs independently of anything you do externally) creates exactly the same kind of bus-fight condition that produces `X`. Use `open` for `emdio` — that leaves it genuinely undriven from the testbench side, letting `leon3mp`'s internal pad behave however it wants without contention, since nothing outside cares about it anymore.

What to do now

1. Replace your current `cpu : leon3mp` positional port map with the named one above — paste it in directly so there's no possibility of a counting error.
2. Re-run `make vcs-launch` and check whether the crash timing changes at all, or whether it disappears.
3. If it still crashes at the same point, the off-by-one wasn't your only issue, and the next most likely thing to check is whatever's actually driving `eth_mac`'s `sram_data => sram_dat` connection at time 0 — but let's isolate this fix first before stacking more hypotheses, since this miscounted port map is a structurally certain bug regardless of whether it's the *only* one.

Run it and tell me exactly what happens — does it still crash, does the timing change, or does it get further before failing?

I will check and let u know but ...Now that I have this new Port map of eth_mac_p where should I do check the ctrl and status reg of the GRETH in VCS DVE?

Good question — but first, a quick but important clarification on your wording: you said "new port map of eth_mac_p," but the port map I just gave you was for `leon3mp` (the CPU-side fix). The `eth_mac_p` port map is the separate one we discussed earlier (the `eth : eth_mac_p port map(...)` block connecting to `proc_addr_p/proc_data_p/...`). Make sure you haven't mixed these up when editing — they're two different instantiations in the same file.

Now, to answer what you're actually asking: **where to find CTRL and STATUS in the DVE waveform hierarchy**, given your new external architecture.

The key structural difference from your old setup

In your **original**, fully-working internal-GRETH test, the hierarchy was simple:

```
testbench > cpu > e1 (grethm) > apbi/apbo
```

Now, with `eth_mac_p` sitting **outside** `leon3mp` as its own top-level instance in the testbench, the hierarchy path is completely different and goes through several more levels:

```
testbench > eth (eth_mac_p instance) > mac0 (eth_mac instance) > bridge0  
(bridge_top) > greth0 (the OG GRETH core)
```

This matches what you showed me earlier in `eth_mac_p.vhd`'s architecture — it instantiates `mac0 : eth_mac`, which internally instantiates `bridge_top0` and the real unmodified `greth0`.

Where to actually click in DVE

In the DVE hierarchy browser (left pane), drill down exactly like this:

```
testbench
└─ eth                               ← your eth_mac_p instance (named "eth" in your port
map)
```

```

└─ mac0          ← eth_mac instance inside eth_mac_p
    └─ bridge_top0 (or whatever you named it inside eth_mac.vhd)
        └─ greth0  ← the actual original GRETH core
            └─ apbi  ← APB slave input record (contains paddr,
pdata, psel, penable)
                └─ apbo ← APB slave output record (contains prdata)

```

Once you're inside `greth0`, look for these specific signals on the APB side (matching the GRETH register map you already know from your C test):

```

greth0.apbi.paddr  ← which register address is being accessed
greth0.apbi.pdata  ← data being written (for WR() calls)
greth0.apbi.psel   ← chip select going active
greth0.apbi.penable ← APB enable phase
greth0.apbo.prdata ← data being read back (for RD() calls)

```

CTRL is at offset `0x00` and STATUS at `0x04` from GRETH's base — so when you see `paddr` showing `0x00` with `pwrite`/`psel` active, that's your CTRL register access; `0x04` is STATUS.

A faster way if names don't match exactly

Since the exact instance names inside `eth_mac.vhd`/`eth_mac_p.vhd` may differ slightly from what I'm guessing, use DVE's search feature instead of manually drilling down:

1. In DVE, go to the hierarchy/scope browser
2. Use **Edit → Find** (or the search icon) and type `apbi` or `paddr`
3. It will show you every instance in the design hierarchy containing a signal with that name — pick the one nested under your `eth`/`eth_mac_p` instance, not under `cpu`/`leon3mp` (since the internal `grethm` inside `leon3mp` also has `apbi.paddr`, and you want the **external** one now)

One important sanity check before you trust what you see there

Given everything we just found about the `leon3mp` port map miscount, I'd recommend running the simulation **after** applying that fix first, and confirming the crash is resolved (or at least changes), **before** spending time tracing CTRL/STATUS in the waveform — otherwise you might be debugging register behavior on a simulation that's still corrupting itself at time 0 for an unrelated reason, which would make any waveform observation unreliable. Once it boots cleanly past the crash point, then the `greth0.apbi`/`apbo` signals under your `eth` instance will give you trustworthy CTRL/STATUS visibility for Test 1 onward.

So bascially:

Fix the port mapping for leon3mp

Use the new sram1 instance u gave

Fix the portmapping of eth_mac_p in testbench

If i do all this and still it fails to run?

Good summary — that's exactly the three structural fixes on the table. Let me be precise about what each one actually buys you, and equally precise about what happens if you still see a failure after all three, since "fails to run" can mean several very different things at this point and they point to different next steps.

What each fix actually guarantees (and doesn't)

Fix 1 — leon3mp port map (named association, no off-by-one)

This guarantees LEON3 itself boots cleanly with correct, untangled connections to its own SDRAM, PROM, UART, and the (now-disconnected) internal GRETH PHY pins. It does **not** guarantee anything about `eth_mac_p` working — it only removes one specific source of corruption that could crash the CPU before it even gets to your C code.

Fix 2 — dedicated SRAM instance for eth_mac_p (abits=28, own srec file)

This guarantees GRETH (inside `eth_mac_p`) has a private, correctly-sized memory to put its descriptors and frame buffers into. It does **not** guarantee the bus protocol between LEON3 and `eth_mac_p` is correctly timed — that's a separate, still-open question (the `brdyn` /wait-state issue we flagged earlier and never resolved because we don't have `bridge_top`'s internals).

Fix 3 — eth_mac_p port map in testbench

This guarantees the signal names/types line up correctly between the testbench and `eth_mac_p`'s actual entity. It does **not** guarantee the *protocol* on that bus is compatible — a syntactically correct port map can still connect two things that disagree about timing.

So if you do all three and it still fails — here's the systematic order to debug, because "still fails" needs to be split into distinct symptoms

Symptom A — Crashes at the exact same time, same `iuerr`

This means the off-by-one wasn't the (only) cause, or a new positional/typing error was introduced while editing. Action: re-run the same port-counting check I did, line by line, against whatever you actually pasted in — paste me the literal text of what you ran and I'll recount it mechanically rather than trust a transcription.

Symptom B — Crashes later, but still before any printf, or even after some PASS lines

This is real progress — it means LEON3 booted further than before. At this point check the **waveform** specifically at the moment of failure: look at `data` and `address` right before the crash. If `data` shows `X`, you have a live bus-driver conflict (likely the `brdyn` /wait-state mismatch theory) — and that's the point where we genuinely need `bridge_top`'s internal logic, since I cannot diagnose a multi-cycle AHB/APB handshake fix without seeing whether it has any ready/wait mechanism at all.

Symptom C — No crash, simulation runs to completion, but Test 1 fails (`FAIL: GRETH reset bit self-clears` , etc.)

This means the structural wiring is now sound, and you're back into the same kind of register-level debugging we did for your fully-internal GRETH test — except now you also need to confirm `eth_sel` / `proc_ce1` is correctly gating access only when LEON3 is genuinely targeting `0x20000D00`, and that the SRAM you wired up is actually reachable from GRETH's AHB DMA side at the address range GRETH's descriptors expect.

Symptom D — Simulation hangs (no crash, no PASS/FAIL, just stalls)

This is the signature of exactly the `brdyn` -style problem: LEON3 issued a read/write to `eth_mac_p`'s address range and is waiting forever for a ready/acknowledge that never comes, because `bridge_top`'s internal AHB/APB handshake takes multiple cycles and nothing tells LEON3 to wait that long.

What I'd need from you if it's still failing

Rather than stack more hypothetical fixes, the most useful next thing — regardless of which symptom you hit — is the **VCS transcript output exactly as printed**, plus, if it's a crash, the **simulation time at which it crashes now** (compare to the original ~5,000,000 ps). That single comparison alone (did the crash time change at all?) tells us immediately whether Fix 1 had any effect, before we go chasing the SRAM or bus-protocol questions further.

I have not run it yet but for context :

The testbench that u currently have with `eth_mac` file actually ran the test once back then the `leon3mp` port mapping was like this only. The only difference would be that i used randomn singals in the tri-state buffer condition inside the `eth_mac` file. But then since it wasn't working properly(because the `data_st` was always IDLE and the `prdata` was always 0000)HENCE i switched to `eth_mac_p.vhd` which already gave me an inout and then the simulation stop i fixed in the begining of `eth_mac` file again occured in `eth_mac_p` file

This is a really important piece of context that changes my read of the situation. Let me make sure I've got the timeline exactly right before drawing conclusions, because it implies something different from what I'd been assuming.

What you're telling me:

1. You ran `eth_mac.vhd` with this **same** `leon3mp` port map (the one I just flagged as off-by-one) — and it **booted and ran**, no `iuerr` crash. It just didn't work correctly (APB `pready` / `apbo.data_st` stuck at IDLE, `prdata` always `0x0000`).
2. You then switched to `eth_mac_p.vhd` (keeping the same `leon3mp` port map), and **that's when the** `iuerr` **crash started**.

If that's accurate, **my off-by-one finding in the** `leon3mp` **port map is very likely not your actual crash cause** — because the exact same port map ran fine (if incorrectly) with `eth_mac`. That's a useful elimination, and I should have asked for this sequencing before pushing the `leon3mp` fix as the primary suspect. The fix is still worth applying since it's a genuine bug, but it's probably not what's producing the `iuerr`.

This redirects the investigation specifically onto **what's different between** `eth_mac` **and** `eth_mac_p` **'s instantiation**, since that's the one variable that changed exactly when the crash started.

Q: When you switched the testbench from instantiating 'eth_mac' to 'eth_mac_p', what EXACTLY did you change in the instantiation block - just the component name and the _p-suffixed port names, or did you also change which signals (sram_dat, address, data, iosn, etc.) get connected to them?

A: It's identical in structure to the eth_mac one, just renamed signals + _p suffix on ports

Q: Does your eth_mac_p instantiation include a generic map (ABITS, SRAM_BANKS, DATAWIDTH), and if so, are those values identical to what worked-ish with eth_mac, or did something change there too?

A: Yes, exact same generic map (or eth_mac_p doesn't take SRAM_BANKS so I dropped it)

Good — that's a clean, isolated change. So the wiring topology is identical; only the component swapped, with its `_p`-suffixed port names and one fewer generic. That narrows the search to things that are genuinely *different* about how `eth_mac_p` behaves internally compared to `eth_mac`, even when fed the exact same external signals.

The structural difference that matters most

Recall what `eth_mac_p.vhd`'s architecture actually does (document 3) — it doesn't just rename ports, it inserts **real pad components** (`pc3d21`, `pc3o02`, `pc3b04`) between every external `_p` port and the internal `mac0 : eth_mac` instance. Specifically for the signals you're driving from the testbench:

```
vhdl

proc_addr_pad : for i in 0 to ABITS-1 generate
  addr_pad : pc3d21
  port map (PAD => proc_addr_p(i), CIN => mac_proc_addr(i));
end generate;

proc_ce1_pad : pc3d21
  port map (PAD => proc_ce1_p, CIN => mac_proc_ce1);

proc_wen_pad : pc3d21
  port map (PAD => proc_wen_p, CIN => mac_proc_wen);

proc_oen_pad : pc3d21
  port map (PAD => proc_oen_p, CIN => mac_proc_oen);

proc_data_pad : for i in 0 to DATAWIDTH-1 generate
  data_pad : pc3b04
  port map (PAD => proc_data_p(i), I => mac_proc_rdata(i), CIN => mac_proc_wdata(i));
end generate;
```

`pc3d21` and `pc3o02`/`pc3b04` are **physical I/O pad cell models** — these are technology-library primitives meant for real silicon pin behavior (input buffers, output drivers, tristate pads with `OEN` control). In an actual FPGA/ASIC flow, these get bound to real technology cells during synthesis. **In a pure behavioral VCS simulation with no real pad library elaborated underneath them, these components may not have a synthesizable/simulatable body at all** — or if they do (a generic placeholder model), they can introduce **delta-cycle delays, undefined initial states, or X-on-reset behavior** that a plain signal assignment (as in `eth_mac`, which just renames signals directly with no pad in between) would never have.

This is very likely your actual difference: `eth_mac` wires signals directly (works, even if functionally wrong due to the tri-state condition bug you mentioned), while `eth_mac_p` routes everything through pad-cell models that may be unbound, behave with extra delay, or initialize to `X`/undefined — and **that propagating `X` through `proc_addr`/`proc_ce1`/`proc_oen` straight into `bridge_top`'s decode logic at time 0 is a**

very plausible trigger for an immediate `iuerr`, since LEON3's very first reset-vector fetch goes through this exact same shared `data` bus.

I need to verify whether `pc3d21` / `pc3o02` / `pc3b04` actually have working bodies in your simulation library before chasing this further.

Q: Do you have access to the actual VHDL source for `pc3d21`, `pc3o02`, and `pc3b04` (the pad components `eth_mac_p` instantiates)? These would normally be in a techmap/padmap library somewhere in your GRLIB install.

A: Yes, they are real source files but i don't have access to them..they are not the issue because i have used them or another file and no issue occurred then

Q: When VCS compiles `eth_mac_p.vhd`, does the compile log show any warnings about unbound components, missing architecture bodies, or black-boxing for `pc3d21`/`pc3o02`/`pc3b04`?

A: No such warnings appeared

Good — that rules out the pad-library theory cleanly. They're real, bound, working components elsewhere, no compile warnings, so they're not generically broken. My hypothesis there was wrong. Let me drop it rather than keep defending it.

So we're back to: identical wiring topology, identical generics (minus `SRAM_BANKS`), working pad cells — and yet swapping `eth_mac` → `eth_mac_p` is the one change that introduces the crash. That means the difference has to be something **specific to how these particular pads are instantiated inside `eth_mac_p`**, not whether pads work in general.

Let me re-examine `eth_mac_p.vhd`'s actual pad wiring directly against the diagram you sent, since that's the one thing I haven't cross-checked carefully enough yet.

Re-check `eth_mac_p.vhd`.pdf page 4 - the pad instantiation logic >

That came back as raw PDF bytes again — I need to render it as an image properly, same as I did for the testbench PDF.

Ran a command, viewed a file >

Now I can read the exact pad polarity for `pc3d21` precisely, and I found something concrete.

The exact bug — `proc_ce1_pad` polarity is backwards relative to every other input pad

Look very carefully at how `pc3d21` is used everywhere else on this page versus how it's used for `proc_ce1_p`:

```
vhdl

-- Every other pc3d21 input pad on this page:
etxc_pad : pc3d21
    port map (PAD => etx_clk_p, CIN => ethi_s.tx_clk);
erxc_pad : pc3d21
    port map (PAD => erx_clk_p, CIN => ethi_s.rx_clk);
emdint_pad : pc3d21
    port map (PAD => emdint_p, CIN => ethi_s.mdint);

-- proc_ce1_pad, proc_wen_pad, proc_oen_pad, proc_addr_pad:
proc_ce1_pad : pc3d21
    port map (PAD => proc_ce1_p, CIN => mac_proc_ce1);
proc_wen_pad : pc3d21
    port map (PAD => proc_wen_p, CIN => mac_proc_wen);
proc_oen_pad : pc3d21
    port map (PAD => proc_oen_p, CIN => mac_proc_oen);
```

This is actually **consistent** — `PAD` is always the external pin, `CIN` is always the internal captured value. So this part is fine; I was wrong to suspect a polarity flip here, this matches the pattern throughout.

What I actually need to look at: timing through the pad, not polarity

Here's the real distinguishing factor between `eth_mac` and `eth_mac_p` that I can now state precisely. In `eth_mac.vhd`, `proc_ce1` (an entity port) is used **directly** wherever `bridge_top0` needs it — a plain signal, zero delta-cycle delay, value visible the instant the testbench drives it.

In `eth_mac_p.vhd`, the signal path is: testbench drives `proc_ce1_p` → goes through `pc3d21` (an actual component instance, which is a `port map`, meaning **at minimum one delta-cycle delay** even if `pc3d21`'s internal body is a single `CIN <= PAD;` concurrent assignment) → produces `mac_proc_ce1` → **then** feeds into `mac0 : eth_mac`'s `proc_ce1` port → **another** component-boundary delta cycle → finally reaches `bridge_top0` inside `eth_mac`.

Two extra levels of component hierarchy = two extra delta-cycle delays, minimum, before `proc_ce1`/`proc_wen`/`proc_oen`/`proc_addr` become visible to `bridge_top0`'s logic, compared to `eth_mac` where they're visible immediately. If `bridge_top`'s internal logic (which we don't have visibility into) has **any** sensitivity-list-based process that

samples `proc_ce1` synchronously expecting it to align with `clk` in a specific way, or if it does an immediate (zero-delay) combinational decode assuming `proc_addr`/`proc_ce1` are valid the instant `clk` rises, those extra delta cycles from the pad chain can shift signal arrival relative to `clk` edges in a way that breaks decode timing — without ever showing as an `X`, just as a logically wrong/late sample. This is consistent with what you reported for `eth_mac` itself (`prdata` stuck at `0x0000`, `data_st` stuck IDLE) — that's not a crash, that's a timing/protocol mismatch, exactly matching the `brdyn`-less wait-state theory from earlier. **`eth_mac_p` adding more delta-cycle delay on top of an already-marginal timing relationship is very plausibly what tips it over from "wrong but stable" into "X propagation and an iuerr."**

This is exactly the same root cause we kept circling back to — a timing/handshake mismatch on `bridge_top`'s `proc_*` interface — just now with concrete evidence for why `eth_mac_p` makes it worse than `eth_mac` rather than just a hand-wave.

Given this, here's the honest state of things

I cannot give you a precise fix for the timing/handshake issue without `bridge_top`'s actual internal logic — I've said this before and it's still true, and stacking more structural guesses on top of three already-identified-but-unconfirmed issues (leon3mp port count, dedicated SRAM, this delta-cycle timing) risks you spending a lot of VCS runs chasing fixes that don't address the actual gating factor.

Given you haven't run anything yet, my honest recommendation on priority order is:

1. **Apply the leon3mp named port map fix anyway** — it's a confirmed bug regardless of whether it's *the* crash cause, and named association removes it as a variable for good.
2. **Apply the dedicated SRAM fix** — also a confirmed bug, same reasoning.
3. **Run it with `eth_mac` (not `eth_mac_p`) first**, since that's the version that at least boots without crashing, and check whether `prdata`/`data_st` now behave correctly once 1 and 2 are fixed — this isolates whether the timing problem was ever really about `bridge_top`'s protocol, or was actually just a consequence of bugs 1/2 corrupting things upstream.
4. **Only after `eth_mac` works correctly**, move to `eth_mac_p` and see if the extra pad-delay theory actually manifests as a real problem, or if it was moot all along.

This order gets you the most diagnostic information per simulation run, rather than changing three things and one component at once and not knowing which fix mattered.

the sram fix is inside the testbench crct?

Yes — the dedicated SRAM fix goes entirely inside `testbench.vhd`, nowhere else. It's just adding/correcting a `sram` component generate-block instance there, alongside your existing `sram1` / `sbanks` / `prom0` blocks.

To be precise about exactly where and what, here's the complete block again, ready to drop in:

```
vhdl

-- Add this constant near your other file-name constants (promfile, sramfile, sd
constant ethsramfile : string := "ethsram.srec";

-- Add these signal declarations near your other sram_address/sram_dat/sram_ce/s
-- (skip this if you already declared them with the same names)
signal sram_address : std_logic_vector(27 downto 0);
signal sram_dat      : std_logic_vector(31 downto 0);
signal sram_ce, sram_we, sram_oe : std_ulogic;

-- The actual SRAM instance -- this replaces whatever sram1 generate-block
-- you currently have feeding eth_mac/eth_mac_p's private SRAM ports
sram1 : for i in 0 to (sramwidth/8)-1 generate
    sr1 : sram generic map (index => i, abits => 28, tacc => 10, fname => ethsram
        port map (sram_address(27 downto 2), sram_dat(31-i*8 downto 24-i*8),
            sram_ce, sram_we, sram_oe);
end generate;
```

And the empty placeholder file needs to physically exist in whatever directory you run `make vcs-launch` from (same directory as your `systest.srec` / `prom.srec`):

Recreate the empty placeholder ethsram.srec file >

A few things to double check when you drop this in:

Don't duplicate signal names. If your current testbench already has `sram_address` / `sram_dat` / `sram_ce` / `sram_we` / `sram_oe` declared (which it does, per the page 5 image you showed earlier — `signal sram_address : std_logic_vector(27 downto 0);` etc. are already there), don't redeclare them — VHDL will throw a duplicate-declaration error. Just confirm the **width** of `sram_address` is `27 downto 0` (28 bits) and not something narrower, since that's what needs to match `eth_mac`'s `ABITS=28` generic.

The two things that actually change from what you currently have are just `abits => 28` (instead of `sramdepth`, which is 19) and `fname => ethsramfile` (instead of

`sramfile`, which is `"systest.srec"`). Everything else in that generate block stays the same.

Where this connects to `eth_mac` / `eth_mac_p` is purely through the port map — `eth :`
`eth_mac port map(..., sram_addr => sram_address, sram_data => sram_dat, sram_ce1`
`=> sram_ce, sram_wen => sram_we, sram_oen => sram_oe, ...)`, which you already have
 wired up correctly per the page 6 image. So this fix is genuinely self-contained inside
`testbench.vhd` — no changes needed in `eth_mac.vhd`, `eth_mac_p.vhd`, or
`bridge_top.vhd` for this particular issue.

Also as u could see `eth_mac` has different signals like `eth_in` and `eth_on` compared to `eth_mac_p` which had similar names as the signals originally used by `leon3mp...` (but the `eth_in` and `eth_on` are `eth_in_type` and `eth_out_type`)...how should i port map this `eth_mac` (give me along with the signals i need to declare if needed because `phy0` uses the existing signals only)

Good catch — this is a real and separate wiring question from everything else we've covered. Let me trace it precisely.

What `eth_in_type` / `eth_out_type` actually are

From `bridge_pkg` / `custom_eth_pkg` (document 4/5, page 1), these are **records**, not flat signals:

```
vhdl

type eth_in_type is record
    ... (tx_clk, rx_clk, rxd, rx_dv, rx_er, rx_col, rx_crs, mdint, mdio_i, ...)
end record;

type eth_out_type is record
    ... (txd, tx_en, tx_er, mdc, mdio_o, mdio_oe, ...)
end record;
```

`eth_mac.vhd`'s entity declares:

```
vhdl

eth_in  : in  eth_in_type;
eth_out : out eth_out_type;
```

This is a **record-typed port** — a single port that bundles multiple individual signals together. It does **not** match `eth_mac_p`'s flat individual ports (`etx_clk_p`, `erx_clk_p`, `erxd_p`, etc.) at all — they're two structurally different ways of exposing the same PHY interface.

What `phy0` actually drives (the thing you must match)

Looking at testbench.vhd page 6 (already confirmed), `phy0` is instantiated like this:

```
vhdl

p0: phy
  generic map(base1000_t_fd => 0, base1000_t_hd => 0)
  port map(rst, emdio, etx_clk, erx_clk, erxd_t, erx_dv,
           erx_er, erx_col, erx_crs, etxd_t, etx_en, etx_er, emdc, eth_macclk);
```

This is a **positional port map to flat individual signals** — `emdio`, `etx_clk`, `erx_clk`, `erxd_t`, `erx_dv`, `erx_er`, `erx_col`, `erx_crs` (inputs to the PHY model, i.e. outputs from the MAC side it's connected to), and `etxd_t`, `etx_en`, `etx_er`, `emdc` (outputs from the PHY, i.e. inputs to whatever MAC it's wired to).

So `phy0` only knows about flat `std_logic` / `std_logic_vector` signals — it has **no** concept of `eth_in_type` / `eth_out_type` records at all. To connect `eth_mac`'s record-typed `eth_in` / `eth_out` ports to `phy0`'s flat signals, you need to **manually map each individual record field to/from the corresponding flat signal**, since VHDL won't do this conversion automatically — record ports can only be connected to other matching record-typed signals, or you assign field-by-field.

The fix — declare one record signal pair, then bridge field-by-field to `phy0`'s existing flat signals

```
vhdl

-- Declare these record signals (reuse the SAME record types eth_mac.vhd uses)
signal eth_mac_ethi : eth_in_type;
signal eth_mac_etho : eth_out_type;

-- Bridge eth_mac_etho (MAC's outputs) -> phy0's existing flat input signals
-- These concurrent assignments replace driving etxd_t/etx_en/etx_er/emdc from
-- wherever they were driven before (the OLD internal grethm's etho, which you
-- already disconnected/grounded in the leon3mp port map)
etxd_t    <= eth_mac_etho.tx_d;
etx_en    <= eth_mac_etho.tx_en;
etx_er    <= eth_mac_etho.tx_er;
emdc      <= eth_mac_etho.mdc;
emdio     <= eth_mac_etho.mdio_o when eth_mac_etho.mdio_oe = '1' else 'Z';
```



```
-- Bridge phy0's existing flat output signals -> eth_mac_ethi (MAC's inputs)
eth_mac_ethi.tx_clk  <= etx_clk;
eth_mac_ethi.rx_clk  <= erx_clk;
eth_mac_ethi.rxd     <= erxdt;
eth_mac_ethi.rx_dv   <= erx_dv;
eth_mac_ethi.rx_er   <= erx_er;
eth_mac_ethi.rx_col  <= erx_col;
eth_mac_ethi.rx_crs  <= erx_crs;
eth_mac_ethi.mdio_i  <= emdio;
```

Important — check the exact field names against the actual record definition. I'm inferring field names (`txd`, `tx_en`, `mdio_o`, `mdio_oe`, `mdint`, etc.) from how `eth_mac_p.vhd` uses `ethi_s` / `etho_s` in the pad-mapping code you showed me earlier (`ethi_s.tx_clk`, `etho_s.txd(i)`, `etho_s.mdio_o`, `etho_s.mdio_oe`, `ethi_s.mdint`, etc.) — these are the same record types, so the field names should match exactly. Cross-check this against the actual `eth_in_type` / `eth_out_type` declaration in `custom_eth_pkg` / `bridge_pkg` if you have the full package body, since I only have the partial generic record shown in document 4/5 page 1, not the complete field list.

Then the `eth_mac` instantiation itself

```
vhdl

eth : eth_mac
  port map (
    rst      => rst,
    clk_25   => clk,
    clk_50   => clk3,
    eth_in   => eth_mac_ethi,
    eth_out  => eth_mac_etho,
    sram_addr => sram_address,
    sram_data => sram_dat,
    sram_ce1  => sram_ce,
    sram_wen  => sram_we,
    sram_oen  => sram_oe,
    proc_addr => address(27 downto 0),
    proc_data => data,
    proc_ce1  => iosn,
    proc_wen  => writen,
    proc_oen  => oen
  );
```

This matches exactly what you already had on page 6 (`eth_in => ethi`, `eth_out => etho`) — the only change is that `ethi` / `etho` need to actually be declared as

`eth_in_type` / `eth_out_type` records (which I've renamed `eth_mac_ethi` / `eth_mac_etho` above to avoid colliding with any other signal named `ethi` / `etho` already in your file — check there isn't already a same-named signal from the old internal-GRETH wiring causing a conflict) and bridged to `phy0`'s flat signals via the field-by-field assignments above, rather than assuming the record type magically matches flat `std_logic` signals.

One thing to verify before you trust this

I don't have the literal `eth_in_type` / `eth_out_type` record declaration with its complete, exact field list and field order — only inferred field names from context clues across two different files. Before compiling, open `custom_eth_pkg` (or wherever it's fully declared) and confirm every field name I used above (`tx_clk`, `rx_clk`, `rx_dv`, `rx_er`, `rx_col`, `rx_crs`, `mdint`, `mdio_i`, `tx_d`, `tx_en`, `tx_er`, `mdc`, `mdio_o`, `mdio_oe`) exists exactly as spelled — a typo here gives a clean compile error (not a silent bug), so it's a fast thing to fix once you see VCS's error message naming the exact missing field.

so basically declare 2 signals of type `eth_in_type` and `eth_out_type` and then assign the values `tx_clk`, `rx_clk`, `rx_dv`, `rx_er`, `rx_col`, `rx_crs`, `mdint`, `mdio_i`, `tx_d`, `tx_en`, `tx_er`, `mdc`, `mdio_o`, `mdio_oe` in their respective fields and then port map it in `eth_mac` (I don't have to touch the `phy0`)

Yes — that's exactly right, and you've summarized the three-part structure correctly. Let me restate it precisely so it's unambiguous as a checklist, with the one caveat that actually matters.

The three parts, confirmed

1. Declare two signals:

```
vhdl

signal eth_mac_ethi : eth_in_type;
signal eth_mac_etho : eth_out_type;
```

2. Assign each field individually — `eth_mac_ethi`'s fields get driven from `phy0`'s existing flat output signals (`etx_clk`, `erx_clk`, `erxdt`, `erx_dv`, `erx_er`, `erx_col`, `erx_crs`, `emdio`), and `eth_mac_etho`'s fields get read to drive `phy0`'s existing flat input signals (`etxdt`, `etx_en`, `etx_er`, `emdc`, `emdio`). This is the field-by-field bridging code from my last message.

3. Port map `eth_mac` using these two record signals as `eth_in => eth_mac_ethi`, `eth_out => eth_mac_etho` — nothing else changes in that instantiation.

4. Leave `phy0`'s own instantiation completely untouched — correct. It still drives/receives the exact same flat signals (`emdio`, `etx_clk`, `erx_clk`, `erxdt`, `erx_dv`, `erx_er`, `erx_col`, `erx_crs`, `etxdt`, `etx_en`, `etx_er`, `emdc`) it always did. You're only inserting a translation layer *between* `phy0` and `eth_mac`, not modifying `phy0` itself.

The one caveat — `emdio` is bidirectional, so it needs care, not a plain assignment

This is the part worth being precise about, since a naive `<=` both ways on the same signal creates a multi-driver conflict.

`emdio` is a single `inout std_logic` signal shared between the MAC and the PHY. The MAC drives it when sending an MDIO command (`mdio_oe='1'`), and the PHY drives it back when responding to a read. You cannot just write `eth_mac_ethi.mdio_i <= emdio`; **and** `emdio <= eth_mac_etho.mdio_o`; as two unconditional concurrent assignments — that would mean `emdio` is being driven by your bridging code at all times, fighting with `phy0`'s own driving of it.

The read side is fine unconditionally:

```
vhdl
eth_mac_ethi.mdio_i <= emdio;  -- always sample whatever's on the wire, this is
```

But the write side must be tristated, only driving `emdio` when the MAC actually wants to:

```
vhdl
emdio <= eth_mac_etho.mdio_o when eth_mac_etho.mdio_oe = '1' else 'Z';
```

This is exactly what I had in my last message — just flagging it again because it's the one line in that whole bridge that isn't a simple direct assignment, and it's worth double-checking you keep the `when ... else 'Z'` condition rather than simplifying it to a plain `<=` by accident when typing it in.

Everything else in your summary is correct — declare the two records, wire each field individually in both directions, port map `eth_mac` with those two records, and don't touch `phy0`.

Also should i be portmapping this after the phy0 portmap or before? will that cause any issue?

Order doesn't matter, and it won't cause any issue. This is one of the genuinely good things about VHDL's concurrent execution model that's worth understanding clearly, since it removes one thing you need to worry about.

Why order is irrelevant here

Everything outside a `process` block in VHDL architecture — component instantiations (`phy0`, `eth`), and concurrent signal assignments (the field-by-field bridging lines) — are **not executed top-to-bottom like C code**. They're all elaborated simultaneously as a flat set of concurrent statements. The simulator doesn't care what line number something appears on; it only cares about the signal dependency graph (what drives what, and in which delta-cycle/clock-edge that propagates).

So whether you write:

```
vhdl

p0: phy port map (...);

signal eth_mac_ethi : eth_in_type;
...
eth : eth_mac port map (...);
```

or the reverse order:

```
vhdl

signal eth_mac_ethi : eth_in_type;
...
eth : eth_mac port map (...);

p0: phy port map (...);
```

— both produce **identical simulation behavior**. The only hard rule in VHDL is that **signal declarations must appear in the architecture's declarative region** (before the `begin` keyword), not mixed in with the concurrent statements after `begin`. So just make sure `signal eth_mac_ethi : eth_in_type;` and `signal eth_mac_etho : eth_out_type;` go up in your declarations block alongside your other `signal` declarations (`sram_address`, `sram_dat`, etc.) — not physically interleaved between `phy0`'s and `eth`'s port maps after `begin`. The instantiations and bridging assignments themselves, anywhere after `begin`, in any order relative to each other, are fine.

One real ordering consideration — but it's not about simulation correctness

The only place "order" matters at all is **readability for you and anyone else reading the file later**. A sensible convention (not a VHDL requirement) is to group things logically: PHY instantiation, then the record-to-flat-signal bridge, then the `eth_mac` instantiation that consumes those records — top to bottom, telling the story of signal flow. That's purely for human clarity, though; VCS will compile and simulate it identically no matter how you order these blocks.

also why do we have to assign like this? Can't we just use the declared signals itself in port map without driving them with these signals? Since they were also declared only?

```
-- Declare these record signals (reuse the SAME record types
eth_mac.vhd uses)
signal eth_mac_ethi : eth_in_type;
signal eth_mac_etho : eth_out_type;
```

Show more

That's a genuinely good question, and the answer comes down to a basic fact about how VHDL types work: `phy0`'s ports and `eth_mac`'s ports are different types, so they **cannot be connected directly to each other at all** — the assignment isn't optional bookkeeping, it's the only mechanism that exists to get data from one to the other.

Why you can't skip this

Look at what each side actually is:

```
vhdl

-- phy0's ports (flat, individual signals):
etx_clk : out std_ulogic;
erxdt   : out std_logic_vector(3 downto 0);
emdc     : in  std_ulogic;
...

-- eth_mac's ports (one bundled record):
eth_in  : in  eth_in_type;    -- a single record containing tx_clk, rx_clk, rxd,
eth_out : out eth_out_type;   -- a single record containing txd, tx_en, mdc, etc
```

`eth_mac_ethi` (declared as `eth_in_type`) is not "the same kind of thing" as `etx_clk` (declared as `std_ulogic`). One is a composite record with many fields bundled inside it; the other is a single bit. **VHDL has no automatic conversion between a record type and a collection of loose individual signals** — you cannot write `eth_mac_ethi <= etx_clk;` because the types don't match at all (you'd be trying to assign a single bit to an entire record), and you cannot port-map `eth_mac`'s `eth_in` directly to `etx_clk` for the same reason — a port map connection requires the types on both sides to be compatible, and a whole record is never compatible with one bit of one of its own fields.

So the *only* way to move a value from `etx_clk` (a flat signal `phy0` drives) into the `tx_clk` field inside the `eth_mac_ethi` record (which `eth_mac` reads) is to explicitly say "take this one field of the record, and assign it this one flat signal's value" — which is exactly what `eth_mac_ethi.tx_clk <= etx_clk;` does. There's no shortcut around this because the two things genuinely aren't the same data structure.

What declaring the signal alone actually gives you

If you only wrote:

```
vhdl

signal eth_mac_ethi : eth_in_type;
signal eth_mac_etho : eth_out_type;
```

and then used them directly in `eth_mac`'s port map **without** the bridging assignments, you would get two signals that exist, are correctly typed for `eth_mac`'s ports, compile fine — but **carry no meaningful value at all**. `eth_mac_ethi` would just sit at its default/uninitialized value (all fields `'0'` / `'U'`, whatever the record's default is) forever, because nothing is driving it. `eth_mac` would think the PHY's clock is always `0`, `rx_dv` is always `0`, MDIO input is always `0` — meaning GRETH would never see a valid clock, never see real RX data, and the MDIO state machine would hang waiting for a response that never changes. Declaration only creates the "container" — it doesn't connect that container's contents to anything happening elsewhere in the design. The assignments are what actually wire the real PHY signals into the record's fields, and the real record's output fields back out to the real flat wires `phy0` is watching.

So this isn't extra/optional plumbing — it's the entire mechanism by which `eth_mac` and `phy0` (which use two structurally incompatible interface styles) actually exchange real, live values during simulation.

But this issue won't occur during `eth_mac_p` ryt? since `eth_mac_p` uses the same signals as `phy0`

Correct — and this is exactly the distinction worth being precise about, since it's the reason `eth_mac` needs this extra bridging layer while `eth_mac_p` doesn't.

Why `eth_mac_p` avoids this entirely

`eth_mac_p`'s entity ports are **flat individual signals from the start** — `etx_clk_p`, `erx_clk_p`, `erxd_p`, `erx_dv_p`, `erx_er_p`, `erx_col_p`, `erx_crs_p`, `emdint_p`, `etxd_p`, `etx_en_p`, `etx_er_p`, `emdc_p`, `emdio_p` — each one a plain `std_ulogic`/`std_logic_vector`, not bundled into a record. That means these ports are **the same fundamental type category** as `phy0`'s ports (`etx_clk`, `erxdt`, `emdc`, `emdio`, etc.), so they can be connected straight across in the port map with simple name-to-name association, no field-extraction step needed:

```
vhdl

eth : eth_mac_p
  port map (
    ...
    etx_clk_p => etx_clk,
    erx_clk_p => erx_clk,
    erxd_p    => erxdt,
    erx_dv_p  => erx_dv,
    erx_er_p  => erx_er,
    erx_col_p => erx_col,
    erx_crs_p => erx_crs,
    emdint_p  => emdint,
    etxd_p    => etxd,
    etx_en_p  => etx_en,
    etx_er_p  => etx_er,
    emdc_p    => emdc,
    emdio_p   => emdio,
    ...
  );
```

This is a direct, one-line-per-signal connection — exactly like the `eth_mac_p` port map I gave you earlier in the conversation, where I wired `proc_addr_p => address`, `sram_data_p => eth_sram_data`, etc. No intermediate record signals, no field-by-field bridging, because there's no record involved anywhere in this chain at all.

Why this happens internally — `eth_mac_p` does the record conversion for you

The reason `eth_mac_p` can offer flat ports while still using `eth_mac` (which needs the record) underneath is that `eth_mac_p.vhd`'s own architecture does **exactly the same**

kind of bridging you were just asking about — just hidden inside `eth_mac_p` itself, between its flat top-level ports and the `eth_in_type` / `eth_out_type` records that its internal `mac0 : eth_mac` instance needs. You actually already saw this in the very first message I wrote analyzing `eth_mac_p.vhd`'s architecture — it declares `signal ethi_s : eth_in_type; signal etho_s : eth_out_type;` and uses pad components (`pc3d21` / `pc3o02` / `pc3b04`) whose `CIN` / `I` connections plug into `ethi_s.tx_clk`, `etho_s.txd(i)`, `etho_s.mdio_o`, etc. — that's the record-to-flat conversion, done with pads instead of plain signal assignments, but structurally the identical idea.

So the conversion work doesn't disappear when you use `eth_mac_p` — it's just already done for you, one layer down, inside a file you don't need to edit. That's the entire value `eth_mac_p` adds over `eth_mac`: it's a wrapper that exposes a flat-signal interface so things like `phy0` (and your raw testbench wiring) can connect to it without you having to write the field-by-field bridge yourself, the way you do need to when working with `eth_mac` directly.

